

ΑΝΑΦΟΡΑ ΕΡΓΑΣΙΑΣ ΡΥΤΗΘΝ

ΑΝΘΙΜΟΣ ΜΑΡΑΓΚΟΣ : 1115493 (2ο ΕΤΟΣ)

ΚΟΥΚΟΥΒΕΣ ΒΛΑΣΙΟΣ : 1115474 (2ο ΕΤΟΣ)

ΑΓΓΕΛΟΣ ΒΟΥΤΥΡΑΣ : 1115516 (2ο ΕΤΟΣ)

Πρώτο σκέλος : Data Collection

Σε αυτό το μέρος της άσκησης συλλέγουμε δεδομένα μέσω API calling, Web Scrapers. Για τα APIs, πολλές πηγές με courses δυστυχώς επέστρεφαν error 403 καθώς χρειαζόντουσαν authentication key. Οι πηγές που έχουμε βάλει στην εργασία βρέθηκαν έπειτα απο πολύ ψάξιμο και trial and error. Η διαδικασία σε κάθε API είναι πολύ απλή. Βάζουμε Headers ώστε να μην μπλοκάρει το site και μας αναγνωρίσει ως bot. Έπειτα ορίζουμε DataFrame (pandas σε αυτό αλλά και σε όλα τα αρχεία) και τα ονόματα των στηλών. Η λογική πίσω απο κάθε κλήση είναι η εξής:

- Καλούμε το URL και βάζουμε το εκάστοτε search term, το οποίο θα καλεστεί από το GUI
- Προφανώς γίνεται έλεγχος κωδικού επιστροφής (π.χ 200, 403, 404)
- Φορτώνουμε σε μια μεταβλητή τα δεδομένα σε μορφή JSON για απλότητα και συμβατότητα
- Μέσω ενός for loop, όποιο πεδίο καταφέρουμε να αποσπάσουμε από το site, το βάζουμε σε μια μεταβλητή και έπειτα στην κατάλληλη στήλη του DataFrame.
- Εμπειρικά καταλάβαμε ότι κάποια πεδία όπως δυσκολία και γλώσσα δεν δίνονταν απο το εκάστοτε site, επομένως τα προσθέσαμε εμείς και τα αλλάζαμε έπειτα στο normalization.
- Όπως ζητήθηκε υπάρχει μήνυμα που εκτυπώνει επιτυχία αποτυχία ανάκτησης δεδομένων

Όσο για τα Scrapers:

- Χρησιμοποιούμε την βιβλιοθήκη BeautifulSoup, καθώς είναι εύκολη και user friendly για να κάνουμε html parsing
- Στην μεταβλητή soup, που αναθέσαμε το αποτέλεσμα του html parsing, ψάχνουμε το script tag που περιέχει τα δεδομένα σε αρχείο JSON
- Αρχικοποιούμε πίνακα στον οποίο θα τοποθετηθούν με append (όπως ζητήθηκε) τα αποτελέσματα ώστε έπειτα να τα βάλουμε στο DataFrame
- Με παρόμοια λογική με τα APIs προσπαθούμε να αντλήσουμε πληροφορίες από ότι πεδίο μας παρέχει το JSON
- Εφαρμόζουμε μια μικρή κανονικοποίηση και τυπώνουμε μετά μήνυμα Success
- ΠΑΡΟΛΑ ΑΥΤΑ, εμπειρικά διαπιστώσαμε ότι το coursera ήθελε data extraction με regex . Ήταν επίσης το μόνο που δεν παρείχε Duration οπότε βάλουμε fall back “N/A”

=====

ΚΩΔΙΚΑΣ

```
=====

import requests

import pandas as pd

#=====

#Σημείωση: Τα 2 πρώτα API ήταν οι μόνες λύσεις που δεν χρειαζόντουσαν authentication, οπότε τα
έβαλα πρώτα για να έχω δεδομένα για να δουλέψω με το normalization και το csv manager

#=====

#μιμηση πραγματικού χρήστη Agent για να μην μπλοκαριστεί το αίτημα
HEADERS = {
    "User-Agent": (
        "Mozilla/5.0 (X11; Linux x86_64) "
        "AppleWebKit/537.36 (KHTML, like Gecko) "
        "Chrome/120.0.0.0 Safari/537.36"
    ),
    "Accept-Language": "en-US,en;q=0.9",
    "Accept": "text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8",
}

#ορισμα στηλων του dataframe
COLUMNS = [
    "Course Title", "Provider / University", "Category",
    "Difficulty Level", "Cost", "Duration", "Teaching Language"
]

#κενο dataframe με τις στηλες που θελουμε για λογους ασφαλους επιστροφης σε περιπτωση λάθους
def empty_dataframe():
```

```

return pd.DataFrame(columns=COLUMNS)

# =====

# API 1 — DEV.to API

# =====

def fetch_devto(search_term):

    print(f"\n[DEV.to] Sending request...")
    url = (
        f"https://dev.to/api/articles"
        f"?tag={search_term}"
        f"&per_page=20"
    )

    #επιστροφή άδειου dataframe σε περίπτωση που το αίτημα αποτυχει για οποιοδήποτε λόγο
    try:
        response = requests.get(url, timeout=10)
    except requests.RequestException as e:
        print(f"[DEV.to] Connection error: {e}")
        return empty_dataframe()

    #ο κωδικός 200 σημαίνει επιτυχία εύρεσης πληροφοριών, οπότε αν δεν είναι 200 υπήρξε error και
    επιστρέφουμε κενό dataframe
    if response.status_code != 200:
        print(f"[DEV.to] API Error: {response.status_code}")
        return empty_dataframe()

```

```
#περιμενουμε data σε μορφη json και τα αποθηκευουμε σε μεταβλητη
data = response.json()

#τα αποτελεσματα της αναζητησης ειναι μεσα στο πεδιο "docs" του json, οποτε τα αποθηκευουμε σε
μεταβλητη

print(f"[DEV.to] Status: Success — {len(data)} results found")

#κενος πινακας να βάλουμε τα αποτελεσματα για να φτιαξουμε το dataframe
cleaned = []

for item in data:
    tags = item.get("tag_list", [])
    # Category από tags
    category = "General"
    for tag in tags:
        if tag in ["python", "javascript", "java", "programming", "webdev"]:
            category = "Programming"
            break
        elif tag in ["datascience", "machinelearning", "ai"]:
            category = "Data Science"
            break
        elif tag in ["sql", "database"]:
            category = "Database"
            break

    title = item.get("title", "")
    if not title or title == "nan":
        continue # παράλειψε άρθρα χωρίς τίτλο
```

```
cleaned.append({
    "Course Title" : title,
    "Provider / University": item.get("user", {}).get("name", "Unknown"),
    "Category" : category,
    "Difficulty Level" : "All Levels",
    "Cost" : "Free",
    "Duration" : "Self-paced",
    "Teaching Language" : "English" # DEV.to είναι αγγλόφωνο site
})
```

```
return pd.DataFrame(cleaned) if cleaned else empty_dataframe()
```

```
# =====
```

```
# API 2 — iTunes Search API
```

```
# =====
```

```
def fetch_itunes(search_term):
```

```
    print(f"\n[iTunes] Sending request...")
```

```
    url = (
```

```
        f"https://itunes.apple.com/search"
```

```
        f"?term={search_term}+course"
```

```
        f"&media=podcast"
```

```
        f"&entity=podcast"
```

```
        f"&limit=20"
```

```

f"&lang=en_us"
)

try:
response = requests.get(url, timeout=10)
except requests.RequestException as e:
print(f"[iTunes] Connection error: {e}")
return empty_dataframe()

if response.status_code != 200:
print(f"[iTunes] API Error: {response.status_code}")
return empty_dataframe()

#=====
#μεχρι και εδω το σκεπτικο ειναι ιδιο
#=====

#αναθετουμε τα δεδομενα που επιστρεφονται σε μορφη json σε μεταβλητη και απο εκει τα
αποτελεσματα της αναζητησης που ειναι μεσα στο πεδιο "results" του json σε μια αλλη μεταβλητη
results = response.json().get("results", [])

print(f"[iTunes] Status: Success — {len(results)} results found")

cleaned = []

for item in results:
cleaned.append({
"Course Title" : item.get("collectionName", "N/A"),
"Provider / University": item.get("artistName", "N/A"),
"Category" : ", ".join(item.get("genres", ["N/A"])),

```

```

"Difficulty Level" : "All Levels",

"Cost" : "Free", #τα podcasts είναι δωρεάν

"Duration" : "Self-paced", #και τα podcasts είναι self-paced

"Teaching Language" : "English"

})

return pd.DataFrame(cleaned) if cleaned else empty_dataframe()

# =====

# API 3 — GitHub Search API

# =====

def fetch_github(search_term):

print(f"\n[GitHub] Sending request...")

url = (
f"https://api.github.com/search/repositories"
f"?q={search_term}+course&topic=education&sort=stars"
f"&sort=stars"
f"&order=desc"
f"&per_page=20"
)

try:
response = requests.get(
url,

```



```

headers={"Accept": "application/vnd.github+json"},
timeout=10
)

except requests.RequestException as e:
    print(f"[GitHub] Connection error: {e}")
    return empty_dataframe()

if response.status_code != 200:
    print(f"[GitHub] API Error: {response.status_code}")
    return empty_dataframe()

results = response.json().get("items", [])

#=====
#μεχρι και εδω το σκεπτικο ειναι ιδιο
#=====

print(f"[GitHub] Status: Success — {len(results)} results found")

cleaned = []

#εκτελουμε εναν βρόχο για να περασουμε τα αποτελεσματα της αναζητησης που ειναι μεσα στο πεδιο
"items" του json
# και να τα καθαρισουμε και να τα μετατρεψουμε στη μορφη που θελουμε για το dataframe
for repo in results:
    name = repo.get("name", "N/A")

```

```

# Χρησιμοποιούμε το όνομα του repository ως τίτλο του μαθήματος, κάνοντας κάποιες βασικές
καθαρίσεις

title = name.replace("-", " ").replace("_", " ").title()

# καθαρίζουμε τον τιτλο απο κολλημενες λέξεις και πολύ μεγάλες λέξεις που δεν μοιάζουν με τίτλο
μαθήματος

if len(title) > 80 or (" " not in title and len(title) > 15):
    continue

cleaned.append({
    "Course Title" : title,
    "Provider / University": repo.get("owner", {}).get("login", "N/A"),
    "Category" : repo.get("language", "Programming"),
    "Difficulty Level" : "All Levels", #δεν παρεχονται πληροφορίες μεσω του API για το επιπεδο δυσκολιας,
    τη διαρκεια και το κοστος
    "Cost" : "Free",
    "Duration" : "Self-paced",
    "Teaching Language" : "English"
})

return pd.DataFrame(cleaned) if cleaned else empty_dataframe()

```

=====

```

"""
web scraping, data extraction, regex, HTML parsing
"""

```

```
import requests #για να κάνουμε αιτήματα HTTP και να πάρουμε το HTML των σελίδων που θέλουμε να κάνουμε scraping
```

```
import pandas as pd #για να δημιουργήσουμε και να διαχειριστούμε τα dataframes που θα περιέχουν τα δεδομένα των μαθημάτων
```

```
import json #για να κάνουμε parsing των JSON δεδομένων που βρίσκονται μέσα σε script tags
```

```
import re #για να κανουμε καθαρισμό των δεδομένων που θα εξάγουμε από το HTML
```

```
from bs4 import BeautifulSoup #για να κάνουμε parsing του HTML
```

#οσα site έχουν επιλεχτει ήταν γιατί ήταν απο τα λίγα που δεν φορτώνουν τα δεδομένα τους με JavaScript, οπότε μπορούμε να τα πάρουμε απευθείας από το HTML χωρίς να χρειάζεται να χρησιμοποιήσουμε εργαλεία όπως το Selenium που προσομοιώνουν browser και μπορούν να χειριστούν JavaScript.

```
HEADERS = {
```

```
"User-Agent": (
```

```
"Mozilla/5.0 (X11; Linux x86_64) "
```

```
"AppleWebKit/537.36 (KHTML, like Gecko) "
```

```
"Chrome/120.0.0.0 Safari/537.36"
```

```
),
```

```
"Accept-Language": "en-US,en;q=0.9",
```

```
"Accept": "text/html,application/xhtml+xml,application/xml;q=0.9, */*;q=0.8",
```

```
}
```

```
COLUMNS = [ "Course Title", "Provider / University", "Category",
```

```
"Difficulty Level", "Cost", "Duration", "Teaching Language" ]
```

```
#συναρτηση για http requests με error handling
```

```
#παρομοια λογικη με την make_request που χρησιμοποιουμε και στα API Collectors
```

```
def make_request(url):
```

```
try:
response = requests.get(url, headers=HEADERS, timeout=20)
if response.status_code == 200:
return response.text
else:
print(f"Request failed with status code: {response.status_code}")
return None
except requests.RequestException as e:
print(f"Request error: {e}")
return None
```

#επιστροφή αδειου dataframe σε περίπτωση αποτυχίας

```
def empty_dataframe():
return pd.DataFrame(columns=COLUMNS)
```

```
# =====
```

#SCRAPER 1 - GeeksForGeeks Courses Page

```
# =====
```

```
def scrape_geeksforgeeks(search_term):
```

```
print(f"[GeeksForGeeks] Scraping for '{search_term}'...")
```

```
url = "https://www.geeksforgeeks.org/courses"
```

```
html = make_request(url)
```

```
#αν δεν βρει το html της σελίδας επιστρέφει κενό dataframe
if not html:

print("[GeeksForGeeks] Failed to retrieve data.")

return empty_dataframe()


soup = BeautifulSoup(html, "html.parser")

#ψαχνουμε το script tag που περιεχει τα δεδομενα σε μορφη json και τα αποθηκευουμε σε μεταβλητη
script_tag = soup.find("script", type="application/ld+json")

if not script_tag:

print("[GeeksForGeeks] No JSON data found.")

return empty_dataframe()


#parsing δεδομενων που βρηκαμε και τα πάμε σε μεταβλητη
data = json.loads(script_tag.string)

#τα αποτελέσματα αυτά που περιέχουν τα μαθήματα είναι μέσα στο πεδίο "itemListElement" του json,
οπότε τα αποθηκεύουμε σε μια μεταβλητη
courses_raw = data.get("itemListElement", [])


ready_courses = []


#τρεχουμε λουπα στα json δεδομενα που βρηκαμε για να τα καθαρισουμε και να τα μετατρεψουμε στη
μορφη που θελουμε για το dataframe

#σε ένα αρχικό στάδιο μόνο καθώς εχουμε ξεχωριστο αρχείο για κανονικοποίηση
for item in courses_raw[:20]:

course = item.get("item", {})
```

```
title = course.get("name", "N/A")

#ο provider δεν είναι πάντα διαθέσιμος, οπότε χρησιμοποιούμε το όνομα του ιστότοπου ως προεπιλογή
provider = course.get("provider", {}).get("name", "GeeksForGeeks")

difficulty = next(
(course.get(key) for key in ["difficultyLevel", "educationalLevel", "skillLevel", "Experience"])
if course.get(key)),
"N/A"
)

# για το κόστος ψάχνουμε πεδίο με τίτλο "offers" που μπορεί να περιέχει πληροφορίες για το κόστος,
αλλά αν δεν υπάρχει βάζουμε "N/A"

offers = course.get("offers", [{}])

price = offers[0].get("price", "N/A") if offers else "N/A"

cost = f"${price}" if price != "N/A" else "N/A"

instance = course.get("hasCourseInstance", [{}])

# για τη διάρκεια ψάχνουμε πεδίο με τίτλο "duration" που μπορεί να περιέχει πληροφορίες για τη
διάρκεια, αλλά αν δεν υπάρχει βάζουμε "N/A"

duration_raw = instance[0].get("courseSchedule", {}).get("duration", "N/A") if instance else "N/A"

# μετατροπή P84D -> "84 days"

if duration_raw != "N/A" and duration_raw.startswith("P"):
duration = duration_raw[1:].replace("D", " days").replace("W", " weeks")
else:
duration = duration_raw

if title != "N/A":
```

```

ready_courses.append({
    "Course Title" : title,
    "Provider / University": provider,
    "Category" : "Programming",
    "Difficulty Level" : difficulty,
    "Cost" : cost,
    "Duration" : duration,
    "Teaching Language" : "English"
})

df = pd.DataFrame(ready_courses) if ready_courses else empty_dataframe()
print(f"[GeeksForGeeks] Status: {'Success' if not df.empty else 'No results'} — {len(df)} courses found")
return df

#=====

#SCRAPER 2 - COURSERA COURSES PAGE

#=====

def scrape_coursera(search_term="python"):

    print(f"[Coursera] Searching for '{search_term}'...")

    #χτίζουμε το url της σελίδας αναζήτησης του Coursera με το search term θέλουμε
    url = f"https://www.coursera.org/search?query={search_term}"

    html = make_request(url)

```

```
if not html:

    print("[Coursera] Failed request")

    return empty_dataframe()


soup = BeautifulSoup(html, "html.parser")


#βρισκουμε όλα τα script tags της σελίδας
scripts = soup.find_all("script")


data = []


#flag να ξερουμε αν βρήκαμε script με json δεδομένα
found_json = False


for script in scripts:

    text = script.get_text()

    #ψάχνουμε το script που περιέχει τα δεδομένα των μαθημάτων σε μορφή json

    if "courseName" in text or "courses" in text.lower():

        found_json = True


# εξαγουμε τον τιτλο με regex
titles = re.findall(r'"name": "(.*?)"', text)


# εξαγουμε το πρώτο όνομα από το array "partnerNames"
providers = re.findall(
    r'"partnerNames": \[(.*?)\]',
    text
)
```



```
#δοκιμη 3 λέξεων για να βρούμε το πεδίο που περιέχει το επίπεδο δυσκολίας
```

```
difficulties = (
```

```
re.findall(r'"difficultyLevel":'+"^"+', text) or
```

```
re.findall(r'"educationalLevel":'+"^"+', text) or
```

```
re.findall(r'"skillLevel":'+"^"+', text)
```

```
)
```

```
#εξαγωγή τιμων της μορφης $...
```

```
prices = re.findall(r'\$'+', text)
```

```
for i, title in enumerate(titles[:20]):
```

```
#αν δεν υπαρχει provider, βάζουμε το site
```

```
provider = (
```

```
providers[i]
```

```
if i < len(providers)
```

```
else "Coursera"
```

```
)
```

```
#αν δεν βρουμε δυσκολια, βάζουμε "N/A"
```

```
difficulty = (
```

```
difficulties[i]
```

```
if i < len(difficulties)
```

```
else "N/A"
```

```
)
```

```
#αν δεν βρουμε τιμη, βάζουμε "Varies"
```

```
cost = prices[i] if i < len(prices) else "Varies"
```

```
#αντικατάσταση unicode για το & που εμφανίζεται σε πολλους titλους μέσω παρατηρήσεων
clean_title = re.sub(r'\\u0026', '&', title)

#=====

#δυστυχως επειδη το coursera φορτωνει δεδομενα με JavaScript dynamically, η requests δεν μπορει να
εξαγει σωστά τον πάροχο

#αυτο θα γινόταν με selenium όμως πιστευω οτι το προγραμμα θα μπλεξει και θα γίνει πολυ πιο αργό
#η διαδικασία ίδια είναι
#=====

data.append({
"Course Title": clean_title,
"Provider / University": provider,
"Category": "Online Course",
"Difficulty Level": difficulty,
"Cost": cost,
"Duration": "N/A",
"Teaching Language": "English"
})

#βρηκαμε το script με τα δεδομενα, δεν χρειάζεται να συνεχίσουμε να ψάχνουμε
break

if not found_json:
print("[Coursera] No embedded JSON found.")

df = (
```

```

pd.DataFrame(data)

if data
else empty_dataframe()
)

print(
f"[Coursera] Status: "
f"{'Success' if not df.empty else 'No results'}"
)

return df

#=====
#SCRAPER 3 = CODECADEMY
#=====

def scrape_codecademy(search_term):

print(f"[Codecademy] Scraping for '{search_term}'...")

url = "https://www.codecademy.com/catalog"
html = make_request(url)

if not html:

print("[Codecademy] Failed to retrieve data.")
return empty_dataframe()

```

```
soup = BeautifulSoup(html, "html.parser")
script_tag = soup.find("script", type="application/ld+json")

if not script_tag:
    print("[Codecademy] No JSON data found.")
    return empty_dataframe()

data = json.loads(script_tag.string)
courses_raw = data.get("itemListElement", [])

#=====
#ίδια λογική με GeeksForGeeks, δηλαδή χρησιμοποιούν JSON-LD
#=====

ready_courses = []

for course in courses_raw[:20]:

    title = course.get("name", "N/A")
    provider = course.get("provider", {}).get("name", "Codecademy")

    #προσπαθούμε να βρούμε το επίπεδο δυσκολίας από το url
    #αν υποδηλώνει κάτι ο τίτλος
    difficulty = course.get("url", "")

    if "beginner" in difficulty or "introduction" in difficulty or "learn-" in difficulty:
        difficulty = "Beginner"

    elif "intermediate" in difficulty:
```

```

difficulty = "Intermediate"

else:

difficulty = "N/A"

offers = course.get("offers", {}) #για το κόστος ψάχνουμε πεδίο με τίτλο "offers" που μπορεί να περιέχει
πληροφορίες για το κόστος

price = offers.get("price", None) #αν υπάρχει συγκεκριμένη τιμή

currency = offers.get("priceCurrency", "") #αν υπάρχει νόμισμα, το προσθέτουμε μπροστά από την τιμή,
αλλιώς αφήνουμε κενό

category = offers.get("category", "N/A")


if price == 0 or price == "0":

cost = "Free" #αν η τιμή είναι 0, τυπωνουμε δωρεάν

elif price:

cost = f"{currency}{price}"

else:

cost = category


#κανονικοποίηση διάρκειας απο PT80H -> 80 hours αλλιως return

instance = course.get("hasCourseInstance", [])

workload = instance[0].get("courseWorkload", "N/A") if instance else "N/A"

if workload.startswith("PT") and workload.endswith("H"):

duration = workload[2:-1] + " hours"

else:

duration = workload


# φιλτραρισμα βάσει search_term

description = course.get("description", "").lower()

if search_term.lower() not in title.lower() and search_term.lower() not in description:

continue

```

```

ready_courses.append({
    "Course Title" : title,
    "Provider / University": provider,
    "Category" : "Programming",
    "Difficulty Level" : difficulty,
    "Cost" : cost,
    "Duration" : duration,
    "Teaching Language" : "English"
})

df = pd.DataFrame(ready_courses) if ready_courses else empty_dataframe()
print(f"[Codecademy] Status: {'Success' if not df.empty else 'No results'} — {len(df)} courses found")
return df

```

=====

Δεύτερο Σκέλος : CSV Management

Εδώ δημιουργήσαμε συναρτήσεις οι οποίες αφαιρετικά αποθηκεύουν και φορτώνουν δεδομένα. Αυτές φάνηκαν ιδιαίτερα χρήσιμες στο recommender, graphs και gui καθώς μπορέσαμε να φορτώσουμε όλο το αρχείο και να το επεξεργαστούμε.

- Χρησιμοποιώντας βιβλιοθήκη os βρίσκουμε το path του αρχείου όπου θέλουμε να αποθηκεύσουμε το csv αλλά και από που να το φορτώσουμε αντίστοιχα.
- Βάζουμε όλους τους απαραίτητους ελέγχους σε περίπτωση που δεν βρεθεί και εφαρμόζουμε dataframe στα δεδομένα για επεξεργασία

ΚΩΔΙΚΑΣ

```
"""
CSV management, data handling, file operations, pandas, csv module
"""

import os

import pandas as pd

from utils.normalizer import normalise_dataframe

# Βρίσκει το path του φακέλου του project και μεσω της os.path.join μπαίνει στον φάκελο data να βάλει
το csv

BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
CSV_FILE = os.path.join(BASE_DIR, "data", "courses_1115493.csv")

#οι στήλες του DataFrame
COLUMNS = [

"Course Title", "Provider / University", "Category",

"Difficulty Level", "Cost", "Duration", "Teaching Language"

]

def save_to_csv(df: pd.DataFrame, filepath: str = CSV_FILE):
#σε κάθε συνάρτηση με df πρέπει να ελέγχουμε αν είναι άδειο
if df.empty:
print("[CSV] No data to save.")
return
```

```
# Normalization πριν την αποθήκευση, την οποία θα ξανακάνουμε για "διπλό-φιλτράρισμα"
#περισσότερες πληροφορίες για την διαδικασία στο αρχείο normalizer.py

df = normalise_dataframe(df)

#ελέγχουμε για την ύπαρξη του φακέλου data ώστε να το διαβάσει και να προσθέσει τα νέα δεδομένα
#αν δεν υπάρχει, τα δεδομένα είναι τα πρώτα και μεταφέρονται όλα στο combined

if os.path.exists(filepath):
    existing = pd.read_csv(filepath)
    combined = pd.concat([existing, df], ignore_index=True)
else:
    combined = df

#αφαίρεση διπλοτύπων βάσει τίτλου μαθήματος και παρόχου
#σε περίπτωση διπλοτύπου κρατάμε το τελευταίο, το νεότερο δηλαδή
combined.drop_duplicates(
    subset=["Course Title", "Provider / University"],
    keep="last",
    inplace=True
)

#αποθήκευση στο csv με encoding="utf-8-sig" για ελληνικά. ίσως είναι λίγο αχρείαστο
#καθώς ψαχνουμε σε site με αγγλικά courses αλλά για παν ενδεχόμενο
combined.to_csv(filepath, index=False, encoding="utf-8-sig")
print(f"[CSV] Saved {len(df)} new rows → total {len(combined)} in '{filepath}'")

#πολύ χρήσιμη συνάρτηση για τα γραφήματα και τον recommender
#το σκεπτικό είναι η εύκολη πρόσβαση στα δεδομένα του αρχείου με αφαιρετικότητα
```



```
def load_from_csv(filepath: str = CSV_FILE) -> pd.DataFrame:
    #όπως προαναφέραμε, χρειάζεται ο έλεγχος άδειου df για την σωστή εκτέλεση και debugging του
    προγράμματος

    if not os.path.exists(filepath):

        #αφήνουμε τέτοια μηνύματα σε όλη την έκταση της εφαρμογής για debugging ώστε να κάνουμε track
        κάθε βήμα οτι εκτελείται σωστά

        print(f"[CSV] File '{filepath}' not found.")

        return pd.DataFrame(columns=COLUMNS)

    df = pd.read_csv(filepath, encoding="utf-8-sig")

    print(f"[CSV] Loaded {len(df)} courses from '{filepath}'")

    return df
```

=====

Τρίτο Σκέλος : Graphs

Χρησιμοποιώντας την βιβλιοθήκη που προτείνετε, την Matplotlib, μπορέσαμε να φτιάξουμε μια συνάρτηση για κάθε γράφημα ώστε να μπορεί έπειτα το GUI να τις καλεί ξεχωριστά. Γενικά σε αυτό το πρότζεκτ προτεραιότητα μας είναι η αφαιρετικότητα οπότε έχουμε δημιουργήσει μια συνάρτηση convert to hours και μια get top providers. Η πρώτη φορτώνει την διάρκεια του εκάστοτε μαθήματος , η οποία μπορεί να είναι ώρες, μέρες, εβδομάδες και την μετατρέπει σε ώρες ώστε να μπορέσουμε να κάνουμε συγκρίσεις με μια ενοποιημένη μονάδα μέτρησης. Η get top providers αξιοποιώντας την ποηγούμενη συνάρτηση αντλεί τα μαθήματα με τις 5 μεγαλύτερες διάρκειες. Αν αυτές οι διεργασίες είχαν γίνει μέσα στις συναρτήσεις των γραφημάτων, θα επικρατούσε χάος και μπέρδεμα. Με αυτό το τρόπο καταφέραμε να οργανώσουμε τον κώδικα μας και να τον κάνουμε πιο εύκολο για debugging

και κατανόηση. Χρησιμοποιούμε επίσης και την load csv ώστε να αντλήσουμε δεδομένα απευθείας απο το τρέχον DataFrame.

ΚΩΔΙΚΑΣ

```
"""
οπτικοποίηση γραφημάτων, βοηθητικά εργαλεία.
charts,trends,visualization,graphs,plotting,matplotlib
"""

import matplotlib.pyplot as plt
import re
from utils.csv_manager import CSV_FILE, load_from_csv

#συναρτηση που μετατρέπει διάρκεια σε ώρες ώστε να γίνει σωστά η σύγκριση μεταξύ ωρών, ημερών,
βδομάδων κλπ

#παρόλα αυτά στο γράφημα θα παρουσιάζονται με τις μονάδες τους τις κανονικές. αυτό γίνεται μόνο
για τη σύγκριση

def convertor_to_hours(duration):
    if not isinstance(duration, str):
        return "None"

    duration = duration.strip().lower()

    #χρησιμοποιείται εσωτερικά για να μετατρέψει διάφορες μορφές διάρκειας σε ώρες για τη συγκριση
και την ομαδοποίηση.

    match = re.search(r"(\d+(\.\d+)?)", duration)

    if not match:
        return None
```

```

number = float(match.group(1))

if "hour" in duration:
    return number

elif "day" in duration:
    return number * 24

elif "week" in duration:
    return number * 24 * 7

elif "month" in duration:
    return number * 24 * 30

elif "year" in duration:
    return number * 24 * 365

elif "self" in duration:
    return None

return None

#μαζεύουμε τους 5 παρόχους με μεγαλύτερη διάρκεια
def get_top_providers(df,top=5):
    if df.empty:
        return []
    df=df.copy()
    df["Hours"]=df["Duration"].apply(convertor_to_hours)
    df=df.dropna(subset=["Hours"])
    df=df.sort_values(by="Hours",ascending=False).head(top)
    return df

#5 μαθήματα με μεγαλύτερη διάρκεια
def bar_chart(top=5):
    df=load_from_csv()

```

```
top_providers = get_top_providers(df,top)

if top_providers.empty:
    print("No data to display.")
    return

fig,ax = plt.subplots(figsize=(10,6))

positions = range(len(top_providers))
ax.bar(positions, top_providers["Hours"], color="green")

#εμφανίζει τους παρόχους στον άξονα x με περιστροφή για καλύτερη ανάγνωση
ax.set_xticks(positions)
ax.set_xticklabels(top_providers["Course Title"], rotation=20, ha="right")

#εμφανίζει παργματικη διάρκεια στον άξονα y
ax.set_yticks(top_providers["Hours"])
ax.set_yticklabels(top_providers["Duration"])

#τιτλος και ετικέτες x και y άξονα αντίστοιχα
ax.set_title(f"Top {top} Providers by Course Duration")
ax.set_xlabel("Course Title")
ax.set_ylabel("Duration")

plt.tight_layout()
plt.show()

#κατανομη μαθηματων βάσει δυσκολίας
#όπως παρατηρείτε αφήνω πολλά debuggers για να βλέπετε τον τρόπο σκέψης μας
def pie_chart():
```

```

df=load_from_csv()

print(f"[DEBUG] Loading from: {CSV_FILE}") # να δούμε ποιο αρχείο φορτώνει
if df.empty:
    print("No data to display.")
    return

counts = (df["Difficulty Level"].value_counts())

fig,ax= plt.subplots(figsize=(8,8))

ax.pie(counts.values, labels=counts.index, autopct="%1.1f%%", startangle=140)

ax.set_title("Distribution of Difficulty Levels")

plt.tight_layout()
plt.show()


#ξαναπαιρνουμε τα 5 μαθήματα με την μεγαλύτερη διάρκεια για να τα συσχετίσουμε με το κόστος να
δουμε τη σχέση κόστους-διάρκειας

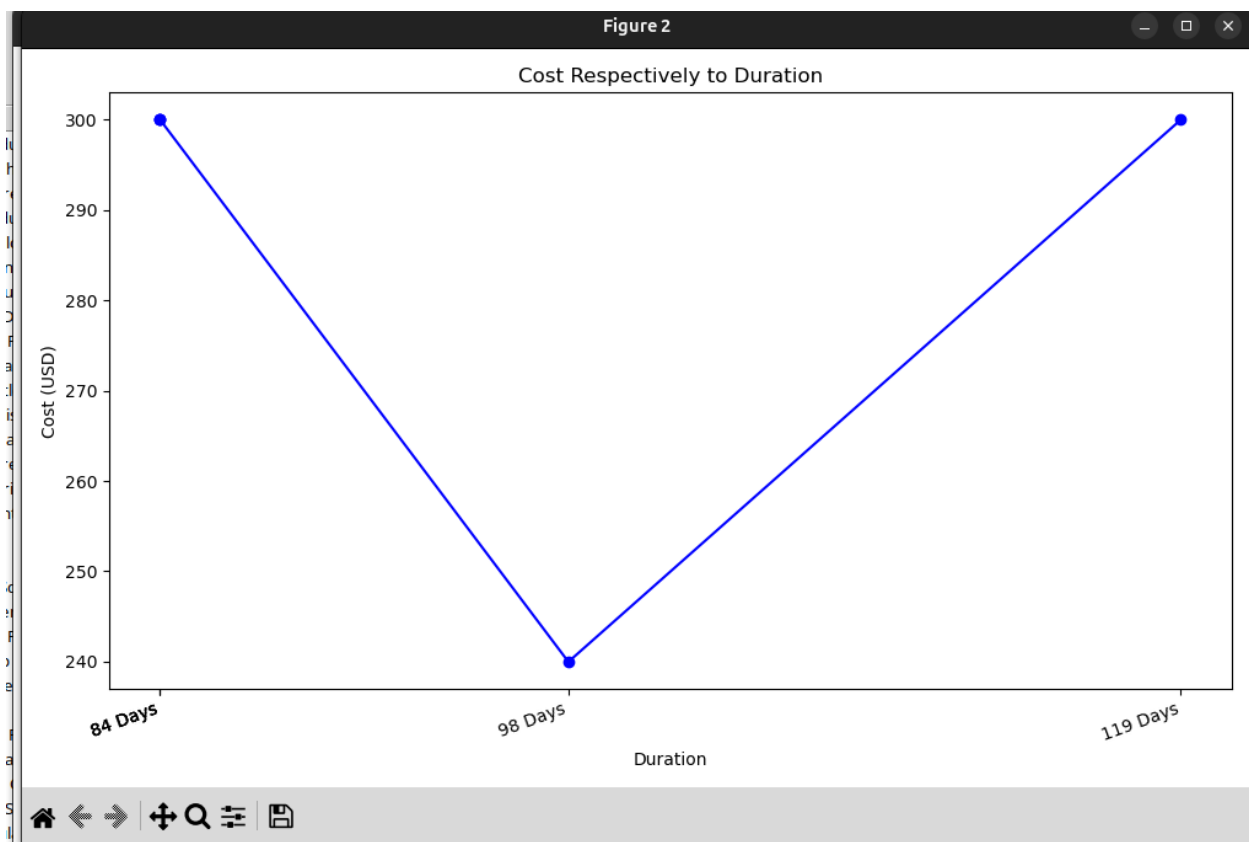
def line_plot():
    df=load_from_csv()
    if df.empty:
        print("No data to display.")
        return
    top5=get_top_providers(df,top=5)
    top5=top5.copy()
    #μεσω lamda function μετατρέπουμε το κόστος απο string σε αριθμό για σύγκριση
    #πάντα σε αντίγραφο για να μην πειράξουμε το αρχικό df
    top5["Cost_Num"]=top5["Cost"].apply(lambda cost: 0.0 if cost.lower() == "free" else
float(re.search(r"(\d+(\.\d+)?)", cost).group(1))
    if re.search(r"(\d+(\.\d+)?)", cost) else None)
    fig,ax=plt.subplots(figsize=(10,6))
    ax.plot(top5["Hours"], top5["Cost_Num"], marker="o", linestyle="-", color="blue")
    ax.set_xticks(top5["Hours"])

```

```
ax.set_xticklabels(top5["Duration"], rotation=20, ha="right")
ax.set_title("Cost Respectively to Duration")
ax.set_xlabel("Duration")
ax.set_ylabel("Cost (USD)")
plt.tight_layout()
plt.show()
```

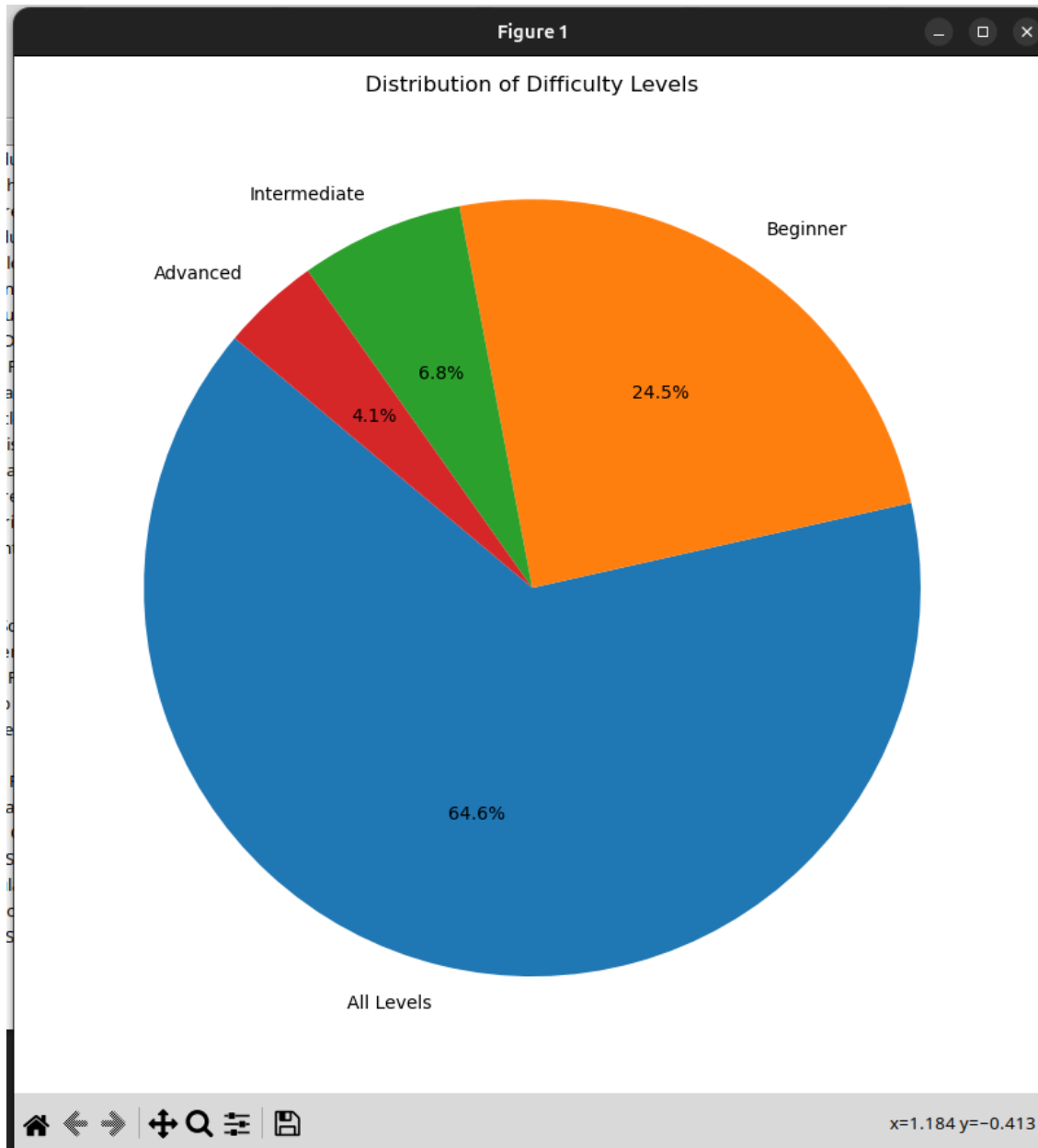
Σχήμα 1:

Κόστος Αγικά με την διάρκεια. Αν δηλαδή ένα χρονοβόρο μάθημα αντιστοιχεί σε μια ψηλή τιμή, το οποίο καταρρίπτεται καθώς μαθημα 98 ημερών είναι πολύ πιο φθηνό από μάθημα 84 ημερών.



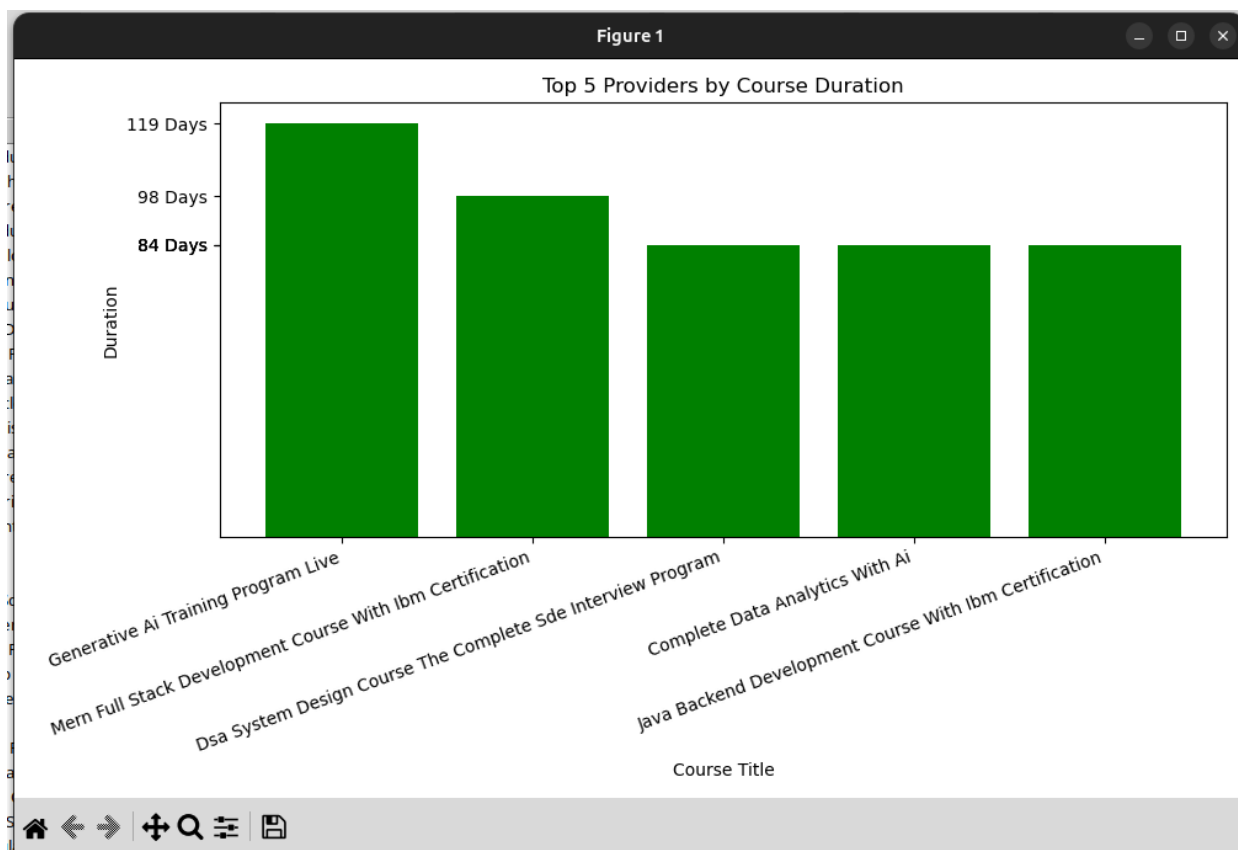
Σχήμα 2:

Κατανομή επιπέδου δυσκολίας μαθημάτων. Όπως φαίνεται, όσο ανεβαίνει η δυσκολία, τόσο μικρότερο το ποσοστό. Τα δύσκολα μαθήματα είναι πιο εξειδικευμένα συνήθως άρα πιο λίγα.



Σχήμα 3: Τα 5 μαθήματα με τη μεγαλύτερη διάρκεια. Βγάζει νόημα καθώς τα θέματα που παρουσιάζονται είναι πολύ δύσκολα και κάποια τάσσονται στην

κατηγορία advanced. Επομένως είναι λογικό δύσκολα μαθήματα να παίρνουν πολύ καιρό όπως Full Stack.



Τέταρτο Σκέλος : Normalizer

Σε αυτό το αρχείο, η καλύτερη λύση που σκεφτήκαμε για mapping ήταν, όπως θα δείτε, οι λίστες. Λίστες οι οποίες έχουν τίτλους category map, difficulty map, irrelevant words, relevant words ώστε να μπορέσουμε να κάνουμε normalize πεδία που δεν μας παρείχε τα data collection section του κώδικα. Πιο συγκεκριμένα έχουμε 2 συναρτήσεις filtering. Η filter results έχει σκοπό να αποτρέψει άσχετα αποτελέσματα που ενδέχεται να συλλέξουν τα API μας όπως podcast, βιβλία, ομιλίες κλπ. Η normalise dataframe εφαρμόζει κάθε συνάρτηση normalise στο εκάστοτε πεδίο και προσέφερε κάποιες επιπλέον προσαρμογές όπως μια έξτρα βοήθεια στη normalise category που για mapping έψαχνε το πεδίο της κατηγορίας αλλά και τον τίτλο βάσει του category map

μας. Αντίστοιχα στο πεδίο της δυσκολίας με το difficulty map. Σε κάθε συνάρτηση κανονικοποίησης χρησιμοποιήσαμε regex για whitespaces, πολλαπλα κενά, αφαίρεση αριθμών και πολλά άλλα όπως θα δείτε και στα σχόλια του κώδικα. Η στην normalise cost αντιμετωπίσαμε θέμα με site που ήθελαν συνδρομή ή δεν παρείχαν τιμή οπότε έπρεπε να εσχεδιάσουμε. Αυτό αποτέλεσε θέμα επίσης στον recommender μετέπειτα. Η κανονικοποίηση της δυσκολίας ήταν επίσης tricky και το map ήταν ο πιο αποτελεσματικός τρόπος να αντιστοιχίσουμε όσο πιο ορθά γινόταν το εκάστοτε μάθημα με τη δυσκολία του.

ΚΩΔΙΚΑΣ

```
"""
κανονικοποίηση δεδομένων όπως:
-φίλτραρισμα τιτλου και df
-αφαίρεση duplicates
-κάνει lowercase
-κάνει κανονικοποίηση μονάδων
-invalid urls/courses
-αφαίρεση punctuation
-αφαίρεση αριθμων
-αφαίρεση whitespace
"""

import re
import pandas as pd
```

#χάρτης κατηγοριών καθώς τα JSON απο τα API και τα Scrapers δεν τα παρέχουν, τα αντλούμε από το πεδίο της κατηγορίας και τον τίτλο

```
CATEGORY_MAP = {  
    "sql": "Databases",  
    "database": "Databases",  
    "data science": "Data Science",  
    "data analytics": "Data Science",  
    "machine learning": "Data Science",  
    "artificial intelligence": "Data Science",  
    "deep learning": "Data Science",  
    "ai": "Data Science",  
    "cybersecurity": "Cybersecurity",  
    "devops": "DevOps",  
    "cloud": "Cloud Computing",  
    "mobile": "Mobile Development",  
    "web development": "Web Development",  
    "sound engineering": "Sound Engineering",  
    "sound": "Sound Engineering",  
    "computer science": "Programming",  
    "web": "Programming",  
    "python": "Programming",  
    "py": "Programming",  
    "programming": "Programming",  
}
```

#το ίδιο ισχύει για την δυσκολία πολλές φορές, οπότε την αντλούμε επίσης απο τον τίτλο

```
DIFFICULTY_MAP = {  
    "beginner": "Beginner",
```

```
"introduction": "Beginner",  
"learn": "Beginner",  
"introductory": "Beginner",  
"intro": "Beginner",  
"what is": "Beginner",  
"basic": "Beginner",  
"tutorial": "Beginner",  
"fundamentals": "Beginner",  
"foundations": "Beginner",  
"intermediate": "Intermediate",  
"medium": "Intermediate",  
"varies": "Intermediate",  
"mixed": "Intermediate",  
"complete": "Intermediate",  
"little experience": "Intermediate",  
"applied": "Intermediate",  
"practical": "Intermediate",  
"computational": "Intermediate",  
"specialization": "Intermediate",  
"analyze": "Intermediate",  
"analysis": "Intermediate",  
"advanced": "Advanced",  
"expert": "Advanced",  
"deep learning": "Advanced",  
"professional": "Advanced",  
"mastering": "Advanced",  
"full stack": "Advanced",  
"devops": "Advanced",  
"all levels": "All Levels"
```

```

}

#λέξεις κλειδιά που θέλουμε και αντίστοιχα δεν θέλουμε να έχει ένα course στον τίτλο του

#έτσι, για να φιλτράρουμε και να συλλέξουμε δεδομένα που μας αφορούν ψάχνουμε για κάποιες λέξεις
οι οποίες μας δείχνουν ότι το δεδομένο που συλλέξαμε είναι εκτός θεματικής της άσκησης

#όλα πάντα βάσει των search terms μας

IRRELEVANT_KEYWORDS = [

"podcast", "novel", "fiction", "policy",

"wellness", "seminar", "lecture", "audioexperience"

]

RELEVANT_KEYWORDS = [

"programming", "python", "java", "javascript", "course",

"coding", "data", "machine learning", "web", "software", "sql"

]

#συνάρτηση που αξιοποιεί τους πίνακες από πάνω για φροντίσει ότι δεν θα συμπεριληφθούν
#άσχετες πληροφορίες στο csv

def filter_results(df, *search_term):

#εφαρμόζουμε lower case στον τίτλο και ψάχνουμε να συμπεριλάβουμε ότι είναι σχετικό

#με τους search terms και τα relevant keywords μας

def is_relevant(title):

title_lower = title.lower()

if any(word in title_lower for word in IRRELEVANT_KEYWORDS):

return False

if any(term.lower() in title_lower for term in search_term):

return True

if any(word in title_lower for word in RELEVANT_KEYWORDS) or any(word in title_lower for word in
CATEGORY_MAP):

```

```

return True

return False

return df[df["Course Title"].apply(is_relevant)]

#ακολουθούν συνάρτησεις κανονικοποίησης κάθε πεδίου

#κανονικοποίηση τίτλου

#δεν καναμε αφαίρεση αριθμών καθώς μπορεί να σηματοδοτούν έκδοση ή νομερο απο σειρά courses
def normalise_course_title(title):
    if not isinstance(title, str) or title.lower() == "nan":
        return "N/A"
    title = title.strip()
    title = re.sub(r"[-./]", " ", title) #αντικατάσταση ειδικών χαρακτήρων με κενό
    title = re.sub(r"^[^w\s]", "", title) #αφαίρεση μη αλφαριθμητικών
    title = re.sub(r"\s+", " ", title) #συμπίεση κενών
    return title.title() #κεφαλαία στο πρώτο γράμμα κάθε λέξης

#κανονικοποίηση ονόματος παρόχου - πανεπιστημίου
def normalise_provider(provider):
    if not isinstance(provider, str):
        return "N/A"
    provider = provider.strip()
    provider = re.sub(r"\s+", " ", provider)
    provider = re.sub(r"^[^w\s]", "", provider)
    provider = re.sub(r"\d+", "", provider) #αφαίρεση αριθμών
    return provider.title()

#κανονικοποίηση κατηγορίας με ψάξιμο ανάμεσα στον χαρτη κατηγοριών πιο πάνω
#για σωστό mapping

```

```

def normalise_category(category):
    if not isinstance(category, str):
        return "General"

    category = category.strip().lower()

    for key, value in CATEGORY_MAP.items():
        if key in category:
            return value

    return "General" #αν δεν βρούμε κατηγορία ούτε από το πεδίο ούτε από τον τίτλο(εξηγω στην
normalise_dataframe), έχουμε fallback = General


def normalise_difficulty(difficulty, title="", url=""):
    #αρχικοποιούμε σε all levels
    base = "All Levels"

    if isinstance(difficulty, str) and difficulty.strip():
        lower = difficulty.strip().lower()
        lower1 = title.strip().lower()

        for keyword, normalized in DIFFICULTY_MAP.items(): #mapping δυσκολίας μετά απο ψάξιμο και σε
        πεδίο δυσκολίας και σε τίτλο για σιγουρία

            if keyword in lower or keyword in lower1: #αυτό δεν μας τρομάζει για error καθώς αποκλείεται να
            έχουν conflicting difficulties

                base = normalized
                break

    #advanced δυσκολία πιο ειδικό ψάξιμο για καλύτερο mapping καθώς είναι πιο σπάνια
    combined = (title + " " + url).lower()

    if any(w in combined for w in ["advanced", "expert", "professional",
    "sde", "mastering", "full stack", "devops"]):
        return "Advanced"

```

```

return base #fallback

def normalise_cost(cost):
    if not isinstance(cost, str):
        return "N/A"
    original = cost.strip()
    lower=original.lower() #τοπική μεταβλητή για να ψάχνουμε το αυτούσιο κόστος
    #καλύπτουμε όλες τις περιπτώσεις του δωρεάν
    if any(w in lower for w in ["free", "0", "no cost", "gratis", "partially free", "enroll for free"]):
        return "Free"
    #εξαγωγή τιμής σε δολάρια
    match = re.search(r"\$(\d+(\.\d+)?)", original)
    if match:
        return match.group()
    if any(w in lower for w in ["paid", "subscription", "premium", "fee"]):
        return "Needs Paid Subscription"

    #fallbacks
    if "varies" in lower:
        return "Varies"

    return "N/A"

def normalise_duration(duration):
    if not isinstance(duration, str):
        return "N/A"
    duration = duration.strip().lower()
    #αν δεν βρεθει αριθμος στο string, θεωρείται self-paced

```

```

match = re.search(r"(\d+(\.\d+)?)", duration)

if not match:
    return "Self-paced"

number = match.group(1)

# mapping της ώρας
if any(w in duration for w in ["hour", "hr"]):
    return f"{number} Hours"
elif "day" in duration:
    return f"{number} Days"
elif any(w in duration for w in ["week", "wk"]):
    return f"{number} Weeks"
elif any(w in duration for w in ["month", "mo"]):
    return f"{number} Months"
elif any(w in duration for w in ["year", "yr"]):
    return f"{number} Years"
return "Self-paced"

#κανονικοποίηση γλώσσας
#pretty straigh-forward, αν υπάρχει συντομογραφία κάποιας γλώσσας την καθαρογράφουμε
def normalise_language(language):
    if not isinstance(language, str):
        return "N/A"
    language = language.strip().lower()
    if "english" in language or "en" in language or "eng" in language:
        return "English"
    elif "spanish" in language or "es" in language or "esp" in language:

```



```

return "Spanish"

elif "french" in language or "fr" in language or "fra" in language:
return "French"

return "Unknown"


def normalise_dataframe(df):
if df.empty:
return df

#στην συναρτήσεις επεξεργασίας dataframe αποφασίσαμε να μην επεξεργαζόμαστε απευθείας το
αρχικό

#αλλά αντί αυτού να φτιάχνουμε ένα αντίγραφο για ασφάλεια

#εφαρμόζουμε όλες τις συναρτήσεις αντίστοιχα για να μην τις καλούμε στα άλλα files ξεχωριστά
df=df.copy()

df["Course Title"]=df["Course Title"].apply(normalise_course_title)

df["Provider / University"]=df["Provider / University"].apply(normalise_provider)

#χρησιμοποιούμε lamda για ευκολα tasks αντί μια ολόκληρη νέα συνάρτηση

#εφαρμόζουμε normalise_category και ψάχνουμε και στο πεδίο κατηγορία και στον τίτλο για καλύτερο
mapping

df["Category"] = df.apply(
lambda row: normalise_category(
str(row["Category"] or "") + " " + str(row["Course Title"] or "")
), axis=1
)

#αντίστοιχα εδώ αντλούμε από τον τίτλο, το πεδίο δυσκολίας και το url
df["Difficulty Level"] = df.apply(
lambda row: normalise_difficulty(
row["Difficulty Level"],
row["Course Title"],
row.get("URL", "")

```

```

),
axis=1
)

df["Cost"]=df["Cost"].apply(normalise_cost)
df["Duration"]=df["Duration"].apply(normalise_duration)
df["Teaching Language"]=df["Teaching Language"].apply(normalise_language)

#ξανα drop duplicates για double-check

df.drop_duplicates(
subset=["Course Title", "Provider / University"],
keep="last",
inplace=True
)

df.reset_index(drop=True, inplace=True) #επαναρύθμιση index μετά την αφαίρεση
return df

```

=====

Πέμπτο Σκέλος : Recommender

Ίσως και το πιο challenging του πρότζεκτ. Η ομαλή λειτουργία αυτού με το GUI ήταν πραγματικό πρόβλημα. Ας εξηγήσουμε όμως τον κώδικα πριν φτάσουμε εκεί. Πρώτο και κυριότερο, ο αλγόριθμος. Σκεφτήκαμε έναν πολύ απλό αλλά αποτελεσματικό αλγόριθμο ο οποίος παίρνει τις 3 βασικές μετρικές ενός course (Διάρκεια, Δυσκολία, Κόστος) και υπολογίζει την αναλογία μεταξύ τους σε μορφή score. Όπως καταλαβαίνετε όσο μεγαλύτερο το score μ τόσο πιο προσιτό το μάθημα. Αυτό το πραγματοποιήσαμε με αρκετές συναρτήσεις όπως:

- Convert cost to number : Μετατροπή του κόστους σε αριθμό.
Προφανώς χρησιμοποιήσαμε regex για την αφαίρεση των συμβόλων και

την μετατροπή σε καθαρο float. Φιάξαμε 2 όμως. Η μια για να μετατρέπει αυτούσιο το ποσό και η άλλη για να μπαίνει στη διαίρεση καθώς διαίρεση με 0 δεν γίνεται.

- Η calculate score η οποία μετατρέπει τα απαραίτητα πεδία σε αριθμούς για να κάνω πράξεις. Μάλιστα, την δυσκολία την κωδικοποιήσαμε ώστε να είναι πιο απλό και εύκολο το mapping.
- Οι υπόλοιπες συναρτήσεις περιγράφονται πλήρως και από τα σχόλια πάνω τον κώδικα

Η δυσκολίες εδώ ήταν 2. Είχαμε σκεφτεί να βάλουμε και search bar σε περίπτωση που ο χρήστης ήθελε συγκεκριμένο μάθημα με επιλογή case sensitive. Το εγκαταλείψαμε αυτό το σχέδιο καθώς αποτέλεσε μεγάλο challenge και τα if conditions γίνανε πολύ μεγάλα. Η δεύτερη δυσκολία ήταν η αποσφαλμάτωση του dataframe σε περίπτωση λάθος, ειδικά με το κόστος. Δυσκολία στον ορισμό του 0 ως free, του None και γενικά του ψαξίματος μέχρι το ανλωτατο όριο αλλά ειδικά με το δωρεάν.

ΚΩΔΙΚΑΣ

```
"""
Recommender system utilities.
```

```
Αλγόριθμος: Duration * Difficulty / max(Cost,1)
```

```
Όσο μεγαλύτερη διάρκεια έχει, τόσο περισσότερο χρόνο δίνει στον χρήστη να μάθει.
```

```
Όσο πιο δύσκολο είναι, τόσο περισσότερο ανταμείβει τον χρήστη και τόσο πιο εξειδικευμένη γνώση του παρέχει.
```

```
Όσο μικρότερο το κόστος, τόσο πιο προσιτό είναι για τον χρήστη.
```

```
Το max(Cost,1) χρησιμοποιείται για να αποφευχθεί η διαίρεση με το 0 στην περίπτωση του free.
```

```
"""
```

```
from utils.graphs import convertor_to_hours
from utils.csv_manager import load_from_csv

import re

import pandas as pd

#αντιστοίχιση δυσκολίας σε αριθμητική τιμή για υπολογισμό score
DIFFICULTY_SCORES = {
    "All Levels": 1,
    "Beginner": 1,
    "Intermediate": 2,
    "Advanced": 3
}

#μεταροπή του cost σε αριθμό. αν δωρεάν τότε επιστρέφουμε 0. αυτό θα χρησιμοποιηθεί στο διάβασμα
του cost filter που μας επιτρέπει το 0

def convert_cost_to_number(cost):
    if not isinstance(cost, str):
        return 0.0

    if "free" in cost.lower():
        return 0.0

    match = re.search(r"(\d+(\.\d+)?)", cost)
    return float(match.group(1)) if match else 0.0

#εδώ πράτουμε ακριβώς το ίδιο απλώς το δωρεάν μεταφράζεται σε 1 για την διαίρεση στον
υπολογισμό του score

def to_cost_score(cost):
    """Για calculate_score — Free = 1.0 για αποφυγή διαίρεσης με 0"""
    if not isinstance(cost, str):
```

```

return 1.0

if "free" in cost.lower():
    return 1.0

match = re.search(r"(\d+(\.\d+)?)", cost)
return float(match.group(1)) if match else 1.0


#υπολογισμός score
def calculate_score(row):
    hours = convertor_to_hours(row["Duration"]) or 0.0
    difficulty = DIFFICULTY_SCORES.get(row["Difficulty Level"],1)
    max_cost = to_cost_score(row["Cost"])
    return (hours * difficulty) / max(max_cost, 1)


#συνάρτηση συστάσεων
def recommend_courses(df=None,Difficulty=None, Category=None, Language=None, max_cost=None,
case_sensitive=False):
    #αν δεν δοθεί DataFrame, φορτώνουμε το ήδη υπάρχον
    #αν δεν υπάρχει ούτε αυτό, πετάει κατάλληλο exception
    if df is None:
        df = load_from_csv()
    print(f"[DEBUG] Total: {len(df)}")
    if df.empty:
        print("No courses available for recommendation.")
    return pd.DataFrame()

    #φιλτράρισμα βάσει επιπέδου δυσκολίας
    if Difficulty and Difficulty.lower() != "all levels" and Difficulty.lower() != "all":
        filtered = pd.DataFrame
    if case_sensitive:
        filtered = df[df["Difficulty Level"] == Difficulty]

```

```

else:

filtered = df[df["Difficulty Level"].str.lower() == Difficulty.lower()]

#αν ο χρήστης βάλει κάποικο level που δεν υπάρχει σε κάποια κατηγορία(π.χ. beginner data science)
#τότε προτείνει από All Levels που τα περιλαμβάνει όλα

if filtered.empty:

print(f"[Recommender] No '{Difficulty}' courses found. Recommending courses for All Levels")

df = df[df["Difficulty Level"] == "All Levels"]

else:

df = filtered

#φιλτράρισμα βάσει δυσκολίας και γλώσσας

if Category and Category.lower() != "all":

if case_sensitive:

df = df[df["Category"] == Category]

else:

df = df[df["Category"].str.lower() == Category.lower()]


if Language and Language.lower() != "all":

if case_sensitive:

df = df[df["Teaching Language"] == Language]

else:

df = df[df["Teaching Language"].str.lower() == Language.lower()]


#στο κόστος βάζουμε όριο που είμαστε να διαθέσουμε και υπολογίζει με ανώτατο όριο το ποσό που
τοποθετήθηκε απο τον χρήστη

#αν το αφήσει κενό θεωρούμε ότι δεν βάζει άρα το κόστος δεν αποτελεί περιορισμό

if max_cost is not None and max_cost>0:

df = df[df["Cost"].apply(convert_cost_to_number) <= max_cost]

print(f"[DEBUG] After cost '{max_cost}': {len(df)}")

elif max_cost is not None and max_cost==0:

```

```

df = df[df["Cost"]=="Free"]

if df.empty and case_sensitive:
    print("[Recommender] No results found. If you want to search in lower case please enable the 'case-insensitive' option.")
    return pd.DataFrame()
elif df.empty:
    return pd.DataFrame()
df = df.copy()

#υπολογισμός score για κάθε μάθημα
df["Score"] = df.apply(calculate_score,axis=1)

#ταξινόμηση και επιλογή των 3 καλύτερων
top3 = df.sort_values("Score", ascending=False).head(3)
top3=top3[["Course Title", "Provider / University",
"Category", "Difficulty Level",
"Cost", "Duration", "Teaching Language", "Score"]]
print("[Recommender] Top 3 courses found.")
return top3

```

=====

Έκτο Σκέλος : App/GUI

Το τελευταίο και πιο σύνθετο κομμάτι της εργασίας ήταν το γραφικό περιβάλλον. Χρησιμοποιήσαμε την βιβλιοθήκη tkinter καθώς είναι η πιο διαδεδομένη για Python GUI και αυτή προτείνεται. Η δομή μας βασίστηκε σε μία κλάση, την CourseGUI, η οποία αρχικοποιείται μέσω constructor και χτίζει το παράθυρο σε τρία τμήματα: κουμπιά, φίλτρα και πίνακας αποτελεσμάτων.

Σκεφτήκαμε ποιά είναι τα πράγματα που θέλουμε να τρέξουν μόλις πατήσουμε compile. Τα κουμπιά, τα δεδομένα (αν υπάρχουν) και τα φίλτρα.

Στον constructor φροντίσαμε να γίνεται αυτόματη φόρτωση του CSV αν υπάρχει ήδη αποθηκευμένο, ώστε ο χρήστης να βλέπει τα δεδομένα χωρίς να χρειάζεται να τα ξανασυλλέξει. Αν το αρχείο δεν βρεθεί, απλώς εκτυπώνεται κατάλληλο μήνυμα στην κονσόλα και ξεκινά καθαρά.

Μια από τις μεγαλύτερες δυσκολίες ήταν το refresh του πίνακα με κάθε νέα συλλογή δεδομένων. Το πρόβλημα ήταν ότι το Treeview widget της tkinter δεν ανανεώνεται αυτόματα, οπότε έπρεπε να σχεδιάσουμε μια συνάρτηση `update_table` η οποία πριν εισάγει τα νέα δεδομένα, διαγράφει ό,τι υπάρχει ήδη στον πίνακα μέσω `get_children()` (όπως καταλαβαίνετε είχε πολύ trial and error αυτό το πρότζεκτ). Με αυτόν τον τρόπο κάθε φορά που καλείται, ο πίνακας σβήνεται και ξαναγεμίζει εκ νέου, αποφεύγοντας duplicates ή παλιά δεδομένα που θα έμεναν ορατά.

Αντίστοιχη δυσκολία συναντήσαμε και στο να χτίσουμε σωστά το ίδιο το Treeview. Έπρεπε να ορίσουμε χειροκίνητα κάθε στήλη, το πλάτος της και την στοίχιση της, ενώ παράλληλα να συνδέσουμε και scrollbar .

Η μεγαλύτερη όμως πρόκληση ήταν το Reset & Restart. Η λογική που θέλαμε ήταν απλή: διαγραφή του παλιού CSV, καθαρισμός του DataFrame, και εκ νέου συλλογή από όλες τις πηγές από την αρχή. Σκεφτήκαμε ότι μπορεί ο χρήστης να θέλει να κάνει refresh, όπως σε μια κανονική εφαρμογή. Στην εκτέλεση όμως αντιμετωπίσαμε αρκετά ζητήματα. Το πρώτο ήταν η διαγραφή του αρχείου, για την οποία χρησιμοποιήσαμε την βιβλιοθήκη `os` και συγκεκριμένα την `os.remove()`, αφού πρώτα ελέγξαμε με `os.path.exists()` αν υπάρχει. Το δεύτερο ήταν ότι έπρεπε να ξανακαλέσουμε όλες τις συναρτήσεις συλλογής, τόσο από APIs όσο και από Scrapers, και να τις ενώσουμε σε ένα νέο DataFrame πριν περάσουν από την `save_and_refresh_data`.

Πονοκέφαλος, ειδικά με όλα τα DataFrame copies. Εδώ αποφασίσαμε να χρησιμοποιήσουμε try/except για κάθε πηγή ξεχωριστά, ώστε αν αποτύχει μία, να μην κρεμάσει ολόκληρη η διαδικασία. Το τρίτο ζήτημα ήταν το

confirmation dialog πριν εκτελεστεί η διαγραφή, το οποίο υλοποιήθηκε με `messagebox.askyesno()` για να αποφευχθεί τυχαία απώλεια δεδομένων.

Γενικά, το GUI ήταν αυτό που έδεσε όλα τα υπόλοιπα κομμάτια μαζί. Κάθε κουμπί αντιστοιχεί σε μία συνάρτηση, κάθε συνάρτηση είναι απομονωμένη και επαναχρησιμοποιήσιμη.

ΚΩΔΙΚΑΣ

```
"""
frontend application, user interface, GUI, tkinter
Αρχείο που περιέχει την κλάση CourseGUI — το γραφικό περιβάλλον της εφαρμογής.
"""

import tkinter as tk
from tkinter import ttk, messagebox
import pandas as pd
import os

from collectors.api_collectors import (fetch_github, fetch_itunes, fetch_devto)
from collectors.scrapers import (scrape_geeksforgeeks, scrape_codecademy, scrape_coursera)
from utils.normalizer import normalise_dataframe, filter_results
from utils.csv_manager import save_to_csv
from utils.graphs import (bar_chart, line_plot, pie_chart)
from utils.recommender import recommend_courses, convert_cost_to_number

class CourseGUI:

    #constructor - οι βασικές λειτουργίες που θέλουμε να γίνουν με το που πατήσουμε το κουμπί του
    compile

    def __init__(self, root):
```

```
# Αποθήκευση αναφοράς στο κύριο παράθυρο της εφαρμογής

self.root = root

# ορισμός τίτλου παραθύρου με τα ονόματα και ΑΜ της ομάδας

self.root.title("ΑΝΘΙΜΟΣ ΜΑΡΑΓΚΟΣ 1115493 - ΑΓΓΕΛΟΣ ΒΟΥΤΥΡΑΣ 1115516 - ΒΛΑΣΙΟΣ ΚΟΥΚΟΥΒΕΣ 1115474")

# Ορισμός διαστάσεων παραθύρου σε pixels (πλάτος x ύψος)

self.root.geometry("1300x800")


# κατασκευή τουμονοπατιού για το αρχείο CSV

# χρησιμοποιούμε abspath+dirname δύο φορές για να παμε στον φάκελο του project και να τι βαλουμε στον επιμέρους φάκελο "data"

self.csv_filename = os.path.join(

os.path.dirname(os.path.abspath(__file__)),

"data", "courses_1115493.csv"

)


# Αρχικοποίηση κενού DataFrame

#θα γεμίσει είτε απο ήδη υπάρχον csv είτε από συλλογη δεδομένων

self.df = pd.DataFrame()


# Λίστα με τους όρους αναζήτησης που χρησιμοποιούνται σε APIs και Scrapers

self.search_terms = [

"programming",

"sound engineering",

"data science",

"sql"

]
```

```

# κλήση των μεθόδων που χτίζουν τα τρία βασικά τμήματα του GUI

self.create_buttons() # δημιουργία κουμπιών

self.create_filters() # δημιουργία φίλτρων

self.create_table() # δημιουργία πίνακα αποτελεσμάτων


# Προσπάθεια αυτόματης φόρτωσης υπάρχοντος CSV κατά την εκκίνηση
#αν υπάρχει ήδη csv στα data,το φορτώνει στον πίνακα
try:

# Ανάγνωση του αρχείου CSV σε DataFrame
self.df = pd.read_csv(self.csv_filename)

# Έλεγχος αν το αρχείο είναι κενό
#αν δεν είναι κενό, φορτώνουμε τα δεδομένα που ήδη έχει αφού κάνουμε update table
#αλλιώς, exception δεν βρέθηκε αρχείο και ξεκινάει το collect
if self.df.empty:
print(f"[{self.csv_filename}] Status: File is empty.")
else:
self.update_table(self.df)
print(f"[{self.csv_filename}] Status: Existing data loaded on startup.")
except FileNotFoundError:
print(f"[{self.csv_filename}] Status: No initial history found.")


# =====
# BUTTONS
# =====

def create_buttons(self):

button_frame = tk.Frame(self.root)
button_frame.pack(pady=10)

```

```
collect_apis_button = tk.Button(
    button_frame,
    text="Collect APIs",
    width=15,
    command=self.collect_api_data # συνδέεται με τη μέθοδο collect_api_data
)

collect_scrapers_button = tk.Button(
    button_frame,
    text="Collect Scrapers",
    width=15,
    command=self.collect_scrapers_data # συνδέεται με τη μέθοδο collect_scrapers_data
)

collect_apis_button.pack(side="left", padx=5)
collect_scrapers_button.pack(side="left", padx=5)

reset_button = tk.Button(
    button_frame,
    text="Reset & Restart",
    width=15,
    fg="red",
    command=self.reset_and_restart
)

reset_button.pack(side="left", padx=5)

export_button = tk.Button(
```

```
button_frame,
text="Export CSV",
width=15,
command=self.export_csv
)
export_button.pack(side="left", padx=5)

bar_chart_button = tk.Button(
button_frame,
text="Bar Chart",
width=15,
command=self.show_bar_chart
)
bar_chart_button.pack(side="left", padx=5)

pie_chart_button = tk.Button(
button_frame,
text="Pie Chart",
width=15,
command=self.show_pie_chart
)
pie_chart_button.pack(side="left", padx=5)

line_plot_button = tk.Button(
button_frame,
text="Line Plot",
width=15,
command=self.show_line_plot
```

```

)

line_plot_button.pack(side="left", padx=5)


# =====
# FILTERS
# =====

def create_filters(self):
    filter_frame = tk.Frame(self.root)
    filter_frame.pack(pady=5)

    # Ετικέτα περιγραφής του φίλτρου
    tk.Label(filter_frame, text="Difficulty").pack(side="left", padx=5)

    # Μεταβλητή που κρατά την επιλογή του χρήστη
    self.difficulty_var = tk.StringVar()
    self.difficulty_box = ttk.Combobox(
        filter_frame,
        textvariable=self.difficulty_var,
        values=["All Levels", "Beginner", "Intermediate", "Advanced"],
        width=18
    )
    self.difficulty_box.pack(side="left", padx=5)

    # Προεπιλεγμένη τιμή: All levels
    self.difficulty_box.set("All Levels")

    tk.Label(filter_frame, text="Category").pack(side="left", padx=5)
    self.category_var = tk.StringVar()

    # Αναπτυσσόμενη λίστα με τις διαθέσιμες κατηγορίες μαθημάτων

```

```
self.category_box = ttk.Combobox(
    filter_frame,
    textvariable=self.category_var,
    values=["All", "Programming", "Data Science", "SQL", "Sound Engineering"],
    width=18
)
self.category_box.pack(side="left", padx=5)
self.category_box.set("All")

tk.Label(filter_frame, text="Language").pack(side="left", padx=5)
self.language_var = tk.StringVar()
self.language_box = ttk.Combobox(
    filter_frame,
    textvariable=self.language_var,
    values=["All", "English", "Spanish", "French"],
    width=18
)
self.language_box.pack(side="left", padx=5)
self.language_box.set("All")

#0 όπως προαναφέραμε σημαίνει δωρεάν
tk.Label(filter_frame, text="Max Cost (0=Free)").pack(side="left", padx=2)
self.cost_entry = tk.Entry(filter_frame, width=10)
self.cost_entry.pack(side="left", padx=5)

recommend_btn = tk.Button(
    filter_frame,
    text="Recommend Courses",
```



```
fg="blue",
command=self.run_recommendation
)
recommend_btn.pack(side="left", padx=10)

#για επιστροφή στο αρχικό μενου των δεδομένων
self.show_all_btn = tk.Button(
filter_frame,
text="Show All",
fg="green",
command=self.show_all_courses
)
self.show_all_btn.pack(side="left", padx=10)

#εμφανίζεται μόνο μετα απο recommendation
self.show_all_btn.pack_forget()

#=====
# REFRESH AFTER RECOMMENDATION
#=====

def show_all_courses(self):
self.update_table(self.df)
self.show_all_btn.pack_forget()

#=====
# TABLE
#=====

def create_table(self):
```

```
#δημιουργουμε container που θα καταλαμβάνει όλο το υπολοιπο παράθυρο
```

```
table_frame = tk.Frame(self.root)
```

```
# fill="both" και expand=True για να μεγαλώνει με το παράθυρο
```

```
table_frame.pack(fill="both", expand=True)
```

```
columns = (
```

```
"Course Title",
```

```
"Provider / University",
```

```
"Category",
```

```
"Difficulty Level",
```

```
"Cost",
```

```
"Duration",
```

```
"Teaching Language"
```

```
)
```

```
#δημιουργία treeview widget
```

```
self.tree = ttk.Treeview(
```

```
table_frame,
```

```
columns=columns,
```

```
show="headings"
```

```
)
```

```
# Ορισμός επικεφαλίδας και πλάτους για κάθε στήλη
```

```
#όνομα στηλης ακριβώς στη μέση
```

```
for col in columns:
```

```
self.tree.heading(col, text=col)
```

```
self.tree.column(col, width=180, anchor="center")
```

```
scrollbar = ttk.Scrollbar(  
    table_frame,  
    orient="vertical",  
    command=self.tree.yview # συνδέεται με την κατακόρυφη κύλιση του Treeview  
)  
  
self.tree.configure(yscrollcommand=scrollbar.set)  
  
scrollbar.pack(side="right", fill="y")  
self.tree.pack(fill="both", expand=True)  
  
#όρισμα το df που θελουμε να εμφανίσουμε στον πίνακα  
  
#αυτοματο refresh του treeview με κάθε κλείσιμο και άνοιγμα της εφαρμογής. σβήνει ότι υπήρχε και  
#ξανατυπώνει εκ νέου  
  
#για αυτο το καλούμε στον constructor  
  
def update_table(self, dataframe):  
    for row in self.tree.get_children():  
        self.tree.delete(row)  
  
    # Αν το DataFrame δεν είναι κενό, εισαγωγή κάθε γραμμής στον πίνακα  
    if dataframe is not None and not dataframe.empty:  
        for _, row in dataframe.iterrows():  
            self.tree.insert(  
                "", "end",  
                values=(  
                    row.get("Course Title", ""),  
                    row.get("Provider / University", ""),  
                    row.get("Category", ""),
```

```

row.get("Difficulty Level", ""),
row.get("Cost", ""),
row.get("Duration", ""),
row.get("Teaching Language", "")
)
)

# =====
# COLLECT DATA
# =====

#αρκετα self-explanatory, μαζευει δεδομένα μέσω API
def collect_api_data(self):
    api_dfs = []

    try:
        df_devto = pd.concat([fetch_devto(term) for term in self.search_terms], ignore_index=True)
        api_dfs.append(df_devto)
        print("[Dev.to API] Status: Success")
    except Exception as e:
        print(f"[Dev.to API] Status: Failed | Error: {e}")

    try:
        df_itunes = pd.concat([fetch_itunes(term) for term in self.search_terms], ignore_index=True)
        api_dfs.append(df_itunes)
        print("[iTunes API] Status: Success")
    except Exception as e:
        print(f"[iTunes API] Status: Failed | Error: {e}")

```

```

try:
df_github = pd.concat([fetch_github(term) for term in self.search_terms], ignore_index=True)
api_dfs.append(df_github)
print("[GitHub API] Status: Success")
except Exception as e:
print(f"[GitHub API] Status: Failed | Error: {e}")

#αν δεν είναι κενός ο πίνακας με τα API data, κάλεσε save and refresh(normalization ξανα γιατί μόνο
καλό κάνει)

if api_dfs:
merged = pd.concat(api_dfs, ignore_index=True)
self.save_and_refresh_data(merged)

def collect_scrapers_data(self):
scraper_dfs = []

try:
df_gfg = pd.concat([scrape_geeksforgeeks(term) for term in self.search_terms], ignore_index=True)
scraper_dfs.append(df_gfg)
print("[GeeksForGeeks Scraper] Status: Success")
except Exception as e:
print(f"[GeeksForGeeks Scraper] Status: Failed | Error: {e}")

try:
df_coursera = pd.concat([scrape_coursera(term) for term in self.search_terms], ignore_index=True)
scraper_dfs.append(df_coursera)

```

```

print("[Coursera Scraper] Status: Success")

except Exception as e:

print(f"[Coursera Scraper] Status: Failed | Error: {e}")


try:

df_codecademy = pd.concat([scrape_codecademy(term) for term in self.search_terms],
ignore_index=True)

scraper_dfs.append(df_codecademy)

print("[Codecademy Scraper] Status: Success")

except Exception as e:

print(f"[Codecademy Scraper] Status: Failed | Error: {e}")


if scraper_dfs:

merged = pd.concat(scraper_dfs, ignore_index=True)

self.save_and_refresh_data(merged)


def save_and_refresh_data(self, new_data_df):

try:

# αρχικοποιούμε κενό DataFrame για το υπάρχον ιστορικό

existing_df = pd.DataFrame()

try:

existing_df = pd.read_csv(self.csv_filename)

except FileNotFoundError:

# Αν δεν υπάρχει αρχείο, συνεχίζουμε με κενό DataFrame

pass


# Εφαρμογή κανονικοποίησης στα νέα δεδομένα (τίτλοι, κόστος, γλώσσα κ.λπ.)

```

```

normalized_new = normalise_dataframe(new_data_df)

# Φιλτράρισμα ώστε να κρατηθούν μόνο σχετικά μαθήματα με τους όρους αναζήτησης
normalized_new = filter_results(normalized_new, *self.search_terms)

print("[Data Normalization] Status: Success")


#αν υπάρχουν παλια δεδομένα ενώνουμε με καινούρια, αλλιώς αυτά είναι τα πρώτα
if not existing_df.empty:
    combined = pd.concat([existing_df, normalized_new], ignore_index=True)
else:
    combined = normalized_new


# αφαίρεση διπλότυπων βάσει τίτλου και παρόχου.ΠΑΛΙ.
#σας εγγυώμαστε, διπλότυπο μέσα στο αρχείο δεν θα βρείτε μετά απο 3 filtrations, στανταρ.
self.df = combined.drop_duplicates(
    subset=["Course Title", "Provider / University"],
    keep="first"
)

# επαναρίθμηση index μετά την αφαίρεση διπλότυπων αλλιώς θα είχαμε 60 στοιχεία αλλα 80
αριθμημένες θέσεις
self.df.reset_index(drop=True, inplace=True)


# Αποθήκευση του τελικού DataFrame στο CSV με υποστήριξη ελληνικών (utf-8-sig)
self.df.to_csv(self.csv_filename, index=False, encoding='utf-8-sig')

print(f"[CSV Manager] Status: Successfully updated {self.csv_filename}")


#ανανεώνουμε treeview με νέα δεδομένα
self.update_table(self.df)

messagebox.showinfo("Success", "Data collected and synchronized successfully!")

```

```
except Exception as e:
    messagebox.showerror("Error", f"Error saving data: {str(e)}")

def run_recommendation(self):
    try:
    try:
        #ο καθιερωμένος μέχρι τώρα έλεγχος για κενό ή ανύπαρκτο csv
        current_df = pd.read_csv(self.csv_filename)
        if current_df.empty:
            messagebox.showwarning("Warning", "The CSV history file is empty. Please collect data first.")
        return
    except FileNotFoundError:
        messagebox.showwarning("Warning", "No CSV history file found. Please collect data first.")
        return

    # διαβασμα τιμών που έβαλε ο χρήστης στα φίλτρα
    difficulty = self.difficulty_var.get()
    category = self.category_var.get()
    language = self.language_var.get()
    cost_text = self.cost_entry.get().strip()
    max_cost = float(cost_text) if cost_text else None

    filtered_df = current_df.copy()

    #ένα ακόμα στρώμα normalization
    #επειδή δεν θέλαμε ο recommender να μην δίνει αποτελέσματα σε ορισμένες περιπτώσεις
```


#αν κάποιος ψαχνει κάποιο level και δεν υπάρχει, επιστρέφει All Levels, όπως μάλλον έχουμε προαναφέρει, μιας και το All Levels τα συμπεριλαμβάνει όλα

```
if category != "All":
```

```
    filtered_df = filtered_df[filtered_df["Category"].str.lower() == category.lower()]
```

```
if language != "All":
```

```
    filtered_df = filtered_df[filtered_df["Teaching Language"].str.lower() == language.lower()]
```

```
if difficulty != "All Levels":
```

```
    temp = filtered_df[filtered_df["Difficulty Level"].str.lower() == difficulty.lower()]
```

```
    if temp.empty:
```

```
        print(f"No '{difficulty}' found. Showing All Levels.")
```

```
    else:
```

```
        filtered_df = temp
```

```
if max_cost is not None and max_cost == 0:
```

```
    filtered_df = filtered_df[filtered_df["Cost"] == "Free"]
```

```
elif max_cost is not None and max_cost > 0:
```

```
    filtered_df = filtered_df[filtered_df["Cost"].apply(convert_cost_to_number) <= max_cost]
```

```
if filtered_df.empty:
```

```
    messagebox.showinfo("No Results", "No matching courses found for the given criteria.")
```

```
self.update_table(pd.DataFrame())
```

```
return
```

```
#υπολογισμός του score για να προτείνουμε κατάλληλα
```

```
recommendations = recommend_courses(df=filtered_df)
```

```
#εξονυχιστικό debugging
```

```
if recommendations is None or recommendations.empty:

    messagebox.showinfo("No Results", "No matching recommendations found.")

else:

    self.update_table(recommendations.head(3))

    self.show_all_btn.pack(side="left", padx=10)


except Exception as e:

    messagebox.showerror("Error", f"Recommendation error: {str(e)}")


#αποθήκευση δεδομένων σε csv με κλασσικούς και basic ελέγχους και διαδικασίες
#προσθέσαμε και messagebox για πιο ωραία και user friendly εικόνα

def export_csv(self):

    try:

        if self.df.empty:

            messagebox.showwarning("Warning", "No data available to export.")

        return

        save_to_csv(self.df)

        messagebox.showinfo("Export Complete", "Current data successfully exported to a new CSV file!")

    except Exception as e:

        messagebox.showerror("Error", str(e))


#κλήσεις συναρτήσεων των γραφημάτων από τα buttons

def show_bar_chart(self):

    try:

        bar_chart(top=5) # top=5 για τα 5 καλύτερα μαθήματα

    except Exception as e:

        messagebox.showerror("Error", str(e))
```

```
def show_pie_chart(self):
    try:
        pie_chart()
    except Exception as e:
        messagebox.showerror("Error", str(e))

def show_line_plot(self):
    try:
        line_plot()
    except Exception as e:
        messagebox.showerror("Error", str(e))

#συνάρτηση επανασυλλογής
def reset_and_restart(self):
    # ζητάει confirmation
    confirm = messagebox.askyesno(
        "Confirmation",
        "Are you sure you want to delete the previous CSV file and restart data collection from scratch?"
    )

    #αν ναι,διαγράφουμε τα παλιά δεδομένα, με την βιβλιοθήκη os γιατι πρόκειται για διαγραφή όλου του
    csv ως αρχείο

    if confirm:
        try:
            if os.path.exists(self.csv_filename):
                os.remove(self.csv_filename)
                print(f"[CSV Manager] Status: Deleted existing file {self.csv_filename}")
            else:
                print(f"[CSV Manager] Status: No file found to delete. Starting fresh.")
```

```

self.df = pd.DataFrame()

self.update_table(self.df)

messagebox.showinfo("Reset Successful", "Previous data cleared. Starting new data collection...")
print("[Reset & Restart] Executing fresh data collection...")

#νέος πίνακας συλλογής σαν τον αντίστοιχο merged πιο πάνω και ακολουθάμε το ίδιο σκεπτικό
#update table με νέο df πιο πάνω, κλησεις συναρτήσεων συλλογής ξανά, ένωση, κανονικοποίηση απο
την save and refresh data και τέλος

#με όλα τα απαραίτητα μηνύματα στη διαδρομή προφανώς
fresh_dfs = []

try:
df_devto = pd.concat([fetch_devto(term) for term in self.search_terms], ignore_index=True)
fresh_dfs.append(df_devto)
except Exception: pass # αγνοούμε αποτυχία μεμονωμένης πηγής

try:
df_itunes = pd.concat([fetch_itunes(term) for term in self.search_terms], ignore_index=True)
fresh_dfs.append(df_itunes)
except Exception: pass

try:
df_github = pd.concat([fetch_github(term) for term in self.search_terms], ignore_index=True)
fresh_dfs.append(df_github)
except Exception: pass

# — Συλλογή από Scrapers —

```

```

try:

df_gfg = pd.concat([scrape_geeksforgeeks(term) for term in self.search_terms], ignore_index=True)

fresh_dfs.append(df_gfg)

except Exception: pass


try:

df_coursera = pd.concat([scrape_coursera(term) for term in self.search_terms], ignore_index=True)

fresh_dfs.append(df_coursera)

except Exception: pass


try:

df_codecademy = pd.concat([scrape_codecademy(term) for term in self.search_terms],
ignore_index=True)

fresh_dfs.append(df_codecademy)

except Exception: pass


if fresh_dfs:

merged_fresh = pd.concat(fresh_dfs, ignore_index=True)

self.save_and_refresh_data(merged_fresh)

else:

messagebox.showwarning("Warning", "Could not fetch any new data after reset.")


except Exception as e:

messagebox.showerror("Error", f"An error occurred during reset: {str(e)}")

```

=====

MAIN

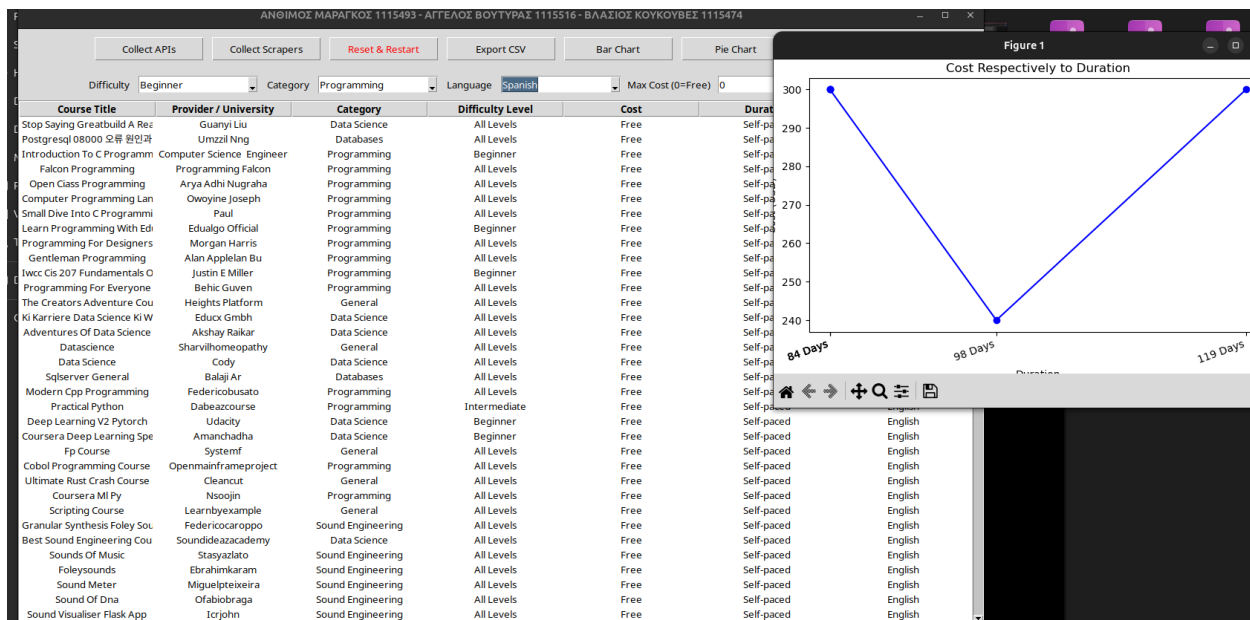
=====

```
import tkinter as tk

from gui.app import CourseGUI # import

if __name__ == "__main__":
    root = tk.Tk()
    app = CourseGUI(root) #αρχικοποίηση κλάσης
    root.mainloop()
```

ΣΤΙΓΜΙΟΤΥΠΑ ΛΕΙΤΟΥΡΓΙΑΣ ΕΦΑΡΜΟΓΗΣ



ΑΝΘΙΜΟΣ ΜΑΡΑΓΚΟΣ 1115493 - ΑΓΓΕΛΟΣ ΒΟΥΤΥΡΑΣ 1115516 - ΒΑΣΙΛΙΟΣ ΚΟΥΚΟΥΒΕΣ 1115474

Collect APIs Collect Scrapers **Reset & Restart** Export CSV Bar Chart Pie Chart Line Plot

Difficulty: Beginner Category: Programming Language: Spanish Max Cost (0=Free): 0 [Recommend Courses](#) [Show All](#)

Course Title	Provider / University	Category	Difficulty Level	Cost	Duration	Teaching Language
Learn Java	Codecademy	Programming	Beginner	Free	17 Hours	English
Learn Javascript	Codecademy	Programming	Beginner	Free	15 Hours	English
Introduction To C Programm	Computer Science Engineer	Programming	Beginner	Free	Self-paced	English

No Results

No matching courses found for the given criteria.

OK

ΑΝΘΙΜΟΣ ΜΑΡΑΓΚΟΣ 1115493 - ΑΓΓΕΛΟΣ ΒΟΥΤΥΡΑΣ 1115516 - ΒΛΑΣΙΟΣ ΚΟΥΚΟΥΒΕΣ 1115474						
Collect APIs Collect Scrapers Reset & Restart Export CSV Bar Chart Pie Chart Line Plot						
Difficulty Beginner Category Programming Language English Max Cost (0=Free) 0 Recommend Courses Show All						
Course Title	Provider / University	Category	Difficulty Level	Cost	Duration	Teaching Language
Learn Java	Codecademy	Programming	Beginner	Free	17 Hours	English
Learn Javascript	Codecademy	Programming	Beginner	Free	15 Hours	English
Introduction To C Programm	Computer Science Engineer	Programming	Beginner	Free	Self-paced	English

ΑΝΘΙΜΟΣ ΜΑΡΑΓΚΟΣ 1115493 - ΑΓΓΕΛΟΣ ΒΟΥΤΥΡΑΣ 1115516 - ΒΛΑΣΙΟΣ ΚΟΥΚΟΥΒΕΣ 1115474						
Collect APIs Collect Scrapers Reset & Restart Export CSV Bar Chart Pie Chart Line Plot						
Difficulty Beginner Category All Language All Max Cost (0=Free) Recommend Courses						
Course Title	Provider / University	Category	Difficulty Level	Cost	Duration	Teaching Language
Computer Programming Lan	Owoyine Joseph	Programming	All Levels	Free	Self-paced	English
Small Dive Into C Programmi	Paul	Programming	All Levels	Free	Self-paced	English
Learn Programming With Ed	Edualgo Official	Programming	Beginner	Free	Self-paced	English
Programming For Designers	Morgan Harris	Programming	All Levels	Free	Self-paced	English
Gentleman Programming	Alan Applelan Bu	Programming	All Levels	Free	Self-paced	English
Iwcc Cis 207 Fundamentals O	Justin E Miller	Programming	Beginner	Free	Self-paced	English
Programming For Everyone	Behic Guven	Programming	All Levels	Free	Self-paced	English
The Creators Adventure Cou	Heights Platform	General	All Levels	Free	Self-paced	English
Ki Karriere Data Science Ki W	Educx Gmbh	Data Science	All Levels	Free	Self-paced	English
Adventures Of Data Science	Akshay Raikar	Data Science	All Levels	Free	Self-paced	English
Datascience	Sharvilhomeopathy	General	All Levels	Free	Self-paced	English
Data Science	Cody	Data Science	All Levels	Free	Self-paced	English
Sqlserver General	Balaji Ar	Database	All Levels	Free	Self-paced	English
Modern Cpp Programming	Federicobusato	Programming	All Levels	Free	Self-paced	English
Practical Python	Dabeazcourse	Programming	All Levels	Free	Self-paced	English
Deep Learning V2 Pytorch	Udacity	Data Science	All Levels	Free	Self-paced	English
Coursera Deep Learning Spe	Amachadha	Data Science	All Levels	Free	Self-paced	English
Fp Course	Systemf	General	All Levels	Free	Self-paced	English
Cobol Programming Course	Openmainframeproject	Programming	All Levels	Free	Self-paced	English
Ultimate Rust Crash Course	Cleancut	General	All Levels	Free	Self-paced	English
Coursera ML Py	Nsoojin	Programming	All Levels	Free	Self-paced	English
Scripting Course	Learnbyexample	General	All Levels	Free	Self-paced	English
Granular Synthesis Foley Sou	Federicocaroppo	Sound Engineering	All Levels	Free	Self-paced	English
Best Sound Engineering Cou	Soundideazacademy	Data Science	All Levels	Free	Self-paced	English
Sounds Of Music	Stasyazlato	Sound Engineering	All Levels	Free	Self-paced	English
Foleysounds	Ebrahimkaram	Sound Engineering	All Levels	Free	Self-paced	English
Sound Meter	Miguelpteixeira	Sound Engineering	All Levels	Free	Self-paced	English
Sound Of Dna	Ofabiabraga	Sound Engineering	All Levels	Free	Self-paced	English
Sound Visualiser Flask App	Icjohn	Sound Engineering	All Levels	Free	Self-paced	English
Sns Paribahan Safe And Sour	Sbsobjb	Sound Engineering	All Levels	Free	Self-paced	English
Commercial Sound Design	Federicocaroppo	Sound Engineering	All Levels	Free	Self-paced	English
Daft Punk Soundboard	Ctailey	Sound Engineering	All Levels	Free	Self-paced	English
Gyro Glove Controlled Vehicl	Stefanocaramagno	Sound Engineering	All Levels	Free	Self-paced	English
Lab 4 Vibe Coding Pingpong	Abhijnanbc	Programming	All Levels	Free	Self-paced	English

ΑΝΘΙΜΟΣ ΜΑΡΑΓΚΟΣ 1115493 - ΑΓΓΕΛΟΣ ΒΟΥΤΥΡΑΣ 1115516 - ΒΛΑΣΙΟΣ ΚΟΥΚΟΥΒΕΣ 1115474

Collect APIs

Collect Scrapers

Reset & Restart

Export CSV

Bar Chart

Pie Chart

Line Plot

Difficulty

Beginner

Category

All

Language

All

Max Cost (0=Free)

Recommend Courses

Course Title	Provider / University	Category	Difficulty Level	Cost	Duration	Teaching Language
Computer Programming Lan	Owoyine Joseph	Programming	All Levels	Free	Self-paced	English
Small Dive Into C Programmi	Paul	Programming	All Levels	Free	Self-paced	English
Learn Programming With Ed	Edualgo Official	Programming	Beginner	Free	Self-paced	English
Programming For Designers	Morgan Harris	Programming	All Levels	Free	Self-paced	English
Gentleman Programming	Alan Applelan Bu	Programming	All Levels	Free	Self-paced	English
Iwcc Cis 207 Fundamentals O	Justin E Miller	Programming	Beginner	Free	Self-paced	English
Programming For Everyone	Behic Guven	Programming	All Levels	Free	Self-paced	English
The Creators Adventure Cou	Heights Platform	General	All Levels	Free	Self-paced	English
Ki Karriere Data Science Ki W	Educo Gmbh	Data Science	All Levels	Free	Self-paced	English
Adventures Of Data Science	Akshay Raikar	Data Science	All Levels	Free	Self-paced	English
Datascience	Sharvilhomeopathy	General	All Levels	Free	Self-paced	English
Data Science	Cody	Data Science	All Levels	Free	Self-paced	English
Sqlserver General	Balaji Ar	Databases	All Levels	Free	Self-paced	English
Modern Cpp Programming	Federicobusato	Programming	All Levels	Free	Self-paced	English
Practical Python	Dabeazcourse	Programming	Intermediate	Free	Self-paced	English
Deep Learning V2 Pytorch	Udacity	Data Science	Beginner	Free	Self-paced	English
Coursera Deep Learning Spe	Amanchadha	Data Science	Beginner	Free	Self-paced	English
Fp Course	Systemf	General	All Levels	Free	Self-paced	English
Cobol Programming Course	Openmainframeproject	Programming	All Levels	Free	Self-paced	English
Ultimate Rust Crash Course	Cleancut	General	All Levels	Free	Self-paced	English
Coursera Ml Py	Nsoojin	Programming	All Levels	Free	Self-paced	English
Scripting Course	Learnbyexample	General	All Levels	Free	Self-paced	English
Granular Synthesis Foley Sou	Federicocaroppo	Sound Engineering	All Levels	Free	Self-paced	English
Best Sound Engineering Cou	Soundideazacademy	Data Science	All Levels	Free	Self-paced	English
Sounds Of Music	Stasyazlato	Sound Engineering	All Levels	Free	Self-paced	English
Foleysounds	Ebrahimkaram	Sound Engineering	All Levels	Free	Self-paced	English
Sound Meter	Miguelpteixeira	Sound Engineering	All Levels	Free	Self-paced	English
Sound Of Dna	Ofabiobraga	Sound Engineering	All Levels	Free	Self-paced	English
Sound Visualiser Flask App	Icrjohn	Sound Engineering	All Levels	Free	Self-paced	English
Sns Paribahan Safe And Sour	Sbsajib	Sound Engineering	All Levels	Free	Self-paced	English
Commercial Sound Design	Federicocaroppo	Sound Engineering	All Levels	Free	Self-paced	English
Daft Punk Soundboard	Ctailey	Sound Engineering	All Levels	Free	Self-paced	English
Gyro Glove Controlled Vehicl	Stefanocaramagno	Sound Engineering	All Levels	Free	Self-paced	English
Lab 4 Vibe Coding Pingpong	Abhijnanbc	Programming	All Levels	Free	Self-paced	English

ΑΝΘΙΜΟΣ ΜΑΡΑΓΚΟΣ 1115493 - ΑΓΓΕΛΟΣ ΒΟΥΤΥΡΑΣ 1115516 - ΒΛΑΣΙΟΣ ΚΟΥΚΟΥΒΕΣ 1115474

Collect APIs

Collect Scrapers

Reset & Restart

Export CSV

Bar Chart

Pie Chart

Line Plot

Difficulty

All Levels

Category

All

Language

All

Max Cost (0=Free)

Recommend Courses

Reset Successful

Previous data cleared. Starting new data collection...

OK

Βιβλιοθήκες και Εργαλεία

Στην ανάπτυξη της εργασίας χρησιμοποιήσαμε τις παρακάτω βιβλιοθήκες:

pandas — Η βασικότερη βιβλιοθήκη του project. Χρησιμοποιήθηκε σε κάθε αρχείο για την αναπαράσταση και επεξεργασία των δεδομένων μέσω DataFrame. Κάθε συλλογή δεδομένων, κανονικοποίηση, φιλτράρισμα και αποθήκευση περνάει από pandas.

requests — Χρησιμοποιήθηκε για τις κλήσεις στα APIs και τη λήψη του HTML περιεχομένου στους scrapers. Μας επέτρεψε να ορίσουμε headers ώστε να αποφύγουμε τα 403 errors και να προσομοιώσουμε κανονικό browser.

BeautifulSoup (bs4) — Η βιβλιοθήκη που χρησιμοποιήσαμε για HTML parsing στους scrapers. Επιλέχθηκε λόγω της απλότητάς της και της ευκολίας με την οποία εντοπίζει συγκεκριμένα tags μέσα στο HTML, όπως το script tag που περιείχε τα JSON δεδομένα.

matplotlib — Η βιβλιοθήκη για τα γραφήματα, όπως προτάθηκε από την εκφώνηση. Με αυτή υλοποιήσαμε τα τρία γραφήματα (bar chart, pie chart, line plot) ως ανεξάρτητες συναρτήσεις που καλούνται από το GUI.

tkinter — Η βιβλιοθήκη για το γραφικό περιβάλλον, επίσης όπως προτάθηκε. Είναι ενσωματωμένη στην Python οπότε δεν χρειάστηκε εξωτερική εγκατάσταση. Χρησιμοποιήθηκαν widgets όπως Button, Entry, Combobox, Treeview και messagebox.

os — Χρησιμοποιήθηκε για τη διαχείριση αρχείων και μονοπατιών. Συγκεκριμένα για την εύρεση του path του CSV, τον έλεγχο ύπαρξής του και τη διαγραφή του στο Reset & Restart.

re (Regular Expressions ή Regex) — Χρησιμοποιήθηκε εκτενώς στον normalizer και τον recommender για καθαρισμό κειμένου, αφαίρεση συμβόλων, εξαγωγή αριθμών από πεδία κόστους και διάρκειας, καθώς και για την εξαγωγή δεδομένων από το Coursera που δεν παρείχε δομημένο JSON.

json — Χρησιμοποιήθηκε για την ανάγνωση και επεξεργασία των δεδομένων που επέστρεφαν τα APIs και οι scrapers σε μορφή JSON.

Σχόλια και Παραδοχές

Κατά την ανάπτυξη της εργασίας έγιναν ορισμένες παραδοχές και αποφάσεις που αξίζει να αναφερθούν:

Όσον αφορά τις κατηγορίες αναζήτησης, επιλέξαμε τους όρους "programming", "data science", "sql" και "sound engineering" ως αντιπροσωπευτικούς και διαφορετικούς μεταξύ τους, ώστε να καλύψουμε τόσο τεχνικά αντικείμενα όσο και κάτι πιο δημιουργικό.

Όσον αφορά το κόστος, θεωρήσαμε ότι η τιμή 0 ισοδυναμεί με "Free". Μαθήματα που δεν παρείχαν τιμή ή απαιτούσαν συνδρομή χωρίς να αναφέρουν ποσό, κανονικοποιήθηκαν επίσης ως "Free" ή "Paid Subscription" ανάλογα με την περίπτωση, καθώς δεν υπήρχε τρόπος να αντληθεί αξιόπιστη αριθμητική τιμή.

Όσον αφορά τη διάρκεια, το Coursera ήταν η μόνη πηγή που δεν παρείχε αυτό το πεδίο, οπότε τοποθετήσαμε "N/A" ως fallback. Στα γραφήματα τα μαθήματα με N/A διάρκεια εξαιρούνται αυτόματα από τους υπολογισμούς.

Όσον αφορά τη δυσκολία και τη γλώσσα, αρκετές πηγές δεν παρείχαν αυτά τα πεδία καθόλου. Σε αυτές τις περιπτώσεις ορίσαμε προεπιλεγμένες τιμές που στη συνέχεια προσπαθεί να διορθώσει ο normalizer βάσει του τίτλου και της κατηγορίας του μαθήματος.

Όσον αφορά το recommendation, επιλέξαμε να εμφανίζονται τα top 3 αποτελέσματα βάσει score ως η πιο καθαρή και χρήσιμη εμπειρία για τον χρήστη. Περισσότερα αποτελέσματα θα δημιουργούσαν σύγχυση δεδομένου ότι ο πίνακας δεν έχει πολύ χώρο.

Τέλος, παραδοχή που αφορά γενικά το project: λόγω του ότι τα APIs και οι scrapers εξαρτώνται από εξωτερικές πηγές, είναι πιθανό κάποια από αυτά να επιστρέφουν λιγότερα ή διαφορετικά αποτελέσματα ανάλογα με τη χρονική στιγμή εκτέλεσης. Αυτός είναι και ο λόγος που υπάρχει το Reset & Restart, ώστε ο χρήστης να μπορεί να ανανεώνει τα δεδομένα όποτε το κρίνει απαραίτητο.

Ευχαριστούμε!!!